Quantum Fault Tolerance Meets Toom's Contours

Michael Ben-Or David Ponarovsky

June 30, 2026

Abstract

We suggest a fault tolerance scheme in the circuit model that both uses the 4D Toric code and has a constant-depth correction cycle. Until now, in the circuit model, the use of a topological code as the primary encoding and a constant-depth correction cycle were only achieved separately; this is the first work to achieve them simultaneously. In addition, to date, the 4D-Toric code has been proposed as a memory element in the thermal noise model. Our work shows that it is also suitable as a memory in the circuit model. To achieve that, we adopt the sweep rule defined in [KP19], as the main component of the correction cycle, thereby bypassing the need for a hierarchical correction cycle. The proof is based on alternative proofs of Gacs's theorems [BS88], [Gac21], which are distinguished from the standard approaches in that they trace the past that leads to error clusters.

Contents

1 Introduction

1.1 Background

Fault-tolerant constructions take ideal quantum circuits and establish their logical equivalence such that they endure fault occurrences. Their efficiency is typically measured by the amount of additional resources – qubits, computation time, range connectivity, etc. – required by the resulting circuit, beyond the scope of theoretical discussion; those parameters translate into the machine complexity and energy cost required to run the circuits. Minimizing overheads brings quantum computing applications closer to the near future.

Since the introduction of the quantum threshold theorem [AB99], which showed that quantum fault tolerance is in principle feasible, the main effort in subsequent work has focused on reducing the additional qubit overhead. A line of work has led to a constant-factor overhead in qubits, first in the instant-classical computation model [Got14], then in the standard circuit model [Gro19], [FGL18], [NP25]. Yet, although the best qubit overhead was reached, the depth overhead was only slightly improved from poly-logarithmic to logarithmic [NP25]. In fact, it seems that the same means that are used to reduce the qubit overhead are the ones that challenge reducing the depth overhead.

A key concept for reducing qubit overhead is to encode qubits with a high-rate quantum LDPC code. For those codes, except for the control-not, there are no known gates that can be realized in constant depth; furthermore, since the structure of the codes is relatively complicated – based on the skeletons of expander graphs - it seems hard to look for realizations of their logical operators. Without a constant-depth realization of logical operators, any construction that doesn't interfere with the order of the logical operations would admit a non constant-depth overhead.

Contrary to high-rate LDPC codes, topological error codes have a geometrical image that is well understood and, as a result, a wide range of works suggest efficient realization of their logical operators. For example, consider the 3D-color code [Bom15], fold-transversal gates on the surface code [BB24], and realization of the Clifford group on the 2D Toric [GV24].

Another branch of work, whose primary goal was to show that the embedding dimension of the computation can be reduced to 2D [BDL26], and more recently [DST26], uses topological codes as the main error correction code. Those constructions simplify the hardware realization; they suggest that quantum computers can be implemented by nearest-neighbor interactions automata - what is believed to be easier to develop. Yet, as in the threshold theorem [AB99], and in contrast to the works that use high-rate LDPC codes, they rely on a hierarchical correction cycle whose execution time grows with system size. Hence, even though those directions have the potential to overcome the non-constant depth overhead per gate, it seems that the correction cycle imposes a barrier for them to come up with a constant factor in depth-overhead construction.

1.2 Stating Our Results

In this work, we present a fault-tolerance construction that uses the 4D Toric code and has a constant-depth 'correction' cycle. Or in other words, a construction uses a relatively simple code, yet doesn't admit the cost of hierarchical correction cycle.

The computation model we consider is quantum circuits subject to *local-stochastic noise*, where only qubits are subjected to the noise, while classical bits are non-noisy. Classical computation is allowed, and the classical depth matters. Namely, at each time unit, only a constant depth of classical computation can be done. Local stochastic channel is defined to act such that the probability of any subset of locations S (space-time coordinates), which is picked prior to the channel's action, being affected decays exponentially with the subset size.

Definition 1.1 (Local stochastic noise channel)**.** *Let $p \in (0,1)$. We say that the noise channel $\mathcal{N}_\mathcal{E}$ is local stochastic with rate p if for every finite set of locations $S \subset \mathbb{N} \times \mathbb{N}$,*

$$\mathbf{Pr}_{\mathcal{E}\sim\mathcal{N}_\mathcal{E}}\left[S \subset \mathrm{Loc}(\mathcal{E})\right] \leq p^{|S|}.$$

For circuits $\tilde{C}$ and C, we say that $\tilde{C}$ simulates C with probability p if, when one stops a noisy run of $\tilde{C}$ at a midpoint, they can, with probability p, recover by non-noisy operation an 'image' of a midpoint in the running of C, and if stopping at the last time point of $\tilde{C}$ enables recovery of the 'image' of C at the end of its run. We proved the following theorem:

Theorem 1.2 (Informal)**.** *There exists a constant $p_0 \in (0,1)$ such that for any local stochastic noise channel $\mathcal{N}_\mathcal{E}$ with rate $p < p_0$, a quantum circuit C with depth μ and width w, and a big enough integer n, there exists a 'logical equivalence' circuit $\tilde{C}$ with depth $\Theta(\mu n)$ and width $\Theta((w+\mu)n)$ that simulates C with probability:*

$$1 - \Theta\left(poly(w,\mu,n)\right) p^{\Theta(n^{\frac{1}{4}})}$$

Furthermore, $\tilde{C}$ has the following two properties:

1. *After a short initialization stage (at depth $\Theta(\log n)$), the qubits are encoded in the n-length 4D Toric code.*
2. *An error correction cycle takes $O(1)$ depth, and the gates in each correction cycle don't depend on time. In particular, if C is the identity circuit, then after the initialization stage, the layers in $\tilde{C}$ are the same.*

As mentioned earlier, both code simplicity and constant depth have already been achieved: the former as part of the quantum automation line of work, and the latter through the use of high-rate error correction codes. However, this is the first construction to achieve both simultaneously. Table 1 summarizes and compares the different works:

Works Line	Error correction code	Constant-depth correction cycle	Depth/qubit overhead	Ref
Concatenation	Concatenation of fixed length code	✗	$f(x)$	[AB99]
High-rate	Quantum expanders/ good QLDPC	✓	$f(x)$	[Gro19], [FGL18], [NP25]
Quantum Automata	2D-Toric code	✗	$f(x)$	[BDL26], [DST26]
Ours	4D-Toric code	✓	$f(x)$	

Table 1: Comparing different types of works on fault-tolerance.

1.3 Discussions, Applications, and Further Directions

Our result can be understood as follow: when the qubit overhead is not a consideration, it means that the 4D Toric code is as good as the good quantum LDPC / Hyperproduct code. That translates to the two following applications:

1. New approach for quantum fault tolerance: With the assistance of resource states and gate teleportation, we obtain a fault-tolerant construction with logarithmic depth overhead. Our proof is inherently different from prior works, describes in higher resolution the expected error, and might be used as a springboard for a constant depth overhead construction.

2. Memory in the circuit model: Protecting a 4D Toric encoded state for an exponentially long time is achieved by performing the same operation at each time unit, suggesting that the 4D Toric code is a promising candidate for memory in the circuit model. Combined with evidence that the 4D Toric code also functions as memory in thermal noise models [Ali+08], and given its relative simplicity, it appears to be a natural choice for real-world memory applications.

2 Outline of The Proof

In studying fault tolerance, the typical method is to first distinguish between configurations that can be tolerated and those that cannot. For example, in von Neumann-style proofs [Neu56], it is common to track the number of flipped bits; if fewer than half are flipped, the logical states can be recovered. In the work of Aharonov and Ben-Or [AB99], a logical bit at the n level admits a logical error if at least a constant fraction of the $n-1$ level logical bits that compose it admit an error. In the work of [Gro19], a tolerated configuration is one in which all encoded registers can be considered as distributed according to sufficiently weak local stochastic noise. The next step is to show that from a tolerated configuration, we move with high probability to another tolerated configuration. For our goals, each approach presents a challenge: The concatenation method, by definition, requires non-trivial depth computation at each step. Tracking the number of flipped bits fails almost immediately, as topological error correction codes have sublinear distance when a linear number of bits is expected to

be flipped. And preserving the local stochastic noise distribution seems to heavily rely on the expansion properties [Gro19] of the underlying structure, where for topological codes, we expect to forge a 'spatial' dependence.

In contrast, Berman and Simon provided a proof that, instead of tracking the dynamics, considers the entire run as a possible outcome, ruling out improbable runs only at the end [BS88]. We outline their idea informally as follows: a logical bit is represented by $n \times n$ bits set on the vertices of the grid. In each correction cycle, the bits determine their value by taking the majority of their north, north-east, and east neighbors, a rule known as Toom's rule. For simplicity, let's ignore the logical operations. Now, given a run, and in particular the set of space-time locations of faults, consider the following directed graph $\mathcal{H} := (V_{\mathcal{H}}, E_{\mathcal{H}})$ induced by it:

1. If at time t, the bit at vertex v diverges relative to the ideal run, then add (v, t) to the vertices.

2. If at time t, vertex v 'leads' one of its neighbors u to diverge, then connect (v, t) and $(u, t + 1)$ by an 'arrow'.

3. Connect any two vertices in $V_{\mathcal{H}}$, which are connected to a third by 'arrows', by a 'fork'.

Notice that the vertices of $\mathcal{H}$ with no incoming arrows correspond to faults; let's call them leaves. Thus, the probability of each running can be bounded by the number of leaves in the induced graph. Their key lemma states that a cluster of flipped bits with a 'perimeter' s, connected by arrows, induces a subgraph with at least $\Theta(s)$ leaves. In other words, the probability of a connected component (by arrows) being flipped decays exponentially with its perimeter.

The graph $\mathcal{H}$ was later named Toom's contours, a term that, to the best of our knowledge, was coined in [SST26].

We chose to follow their approach. The rest of the section reviews the challenges that we encountered on the way to generalize it and the ideas to overcome them.

The Toric Code, the Sweep-Rule, and the Infinite Plane. The first issue was finding an analog for Toom's rule. We decided to adopt the sweep rule invented by Preskill and Kubica in [KP19], which they introduced as a local rule for any CSS code induced by a simplicial Toric, equipped with a notion of 'positive' and 'negative' directions (in their paper 'future' and 'past'). They also provided a rigorous proof that the iterative non-noisy applications of the sweep rule succeed in decoding faults that were sampled independently. For our purposes, we had to extend their proof for cubical Toric[1]. The main reason for this is that the alignment of the positive/negative directions with the axes direction of $\mathbb{R}^4$ space allows quantifying the 'perimeter' of error clusters. Yet, we are sure that our proof can be adjusted to support simplicial Toric complexes as well, and we leave it for further work. In the formal section, we define and prove the claims for Torices in any dimension greater than two, except for

[1]The standard Toric codes.

Claim 3.53, which we proved only to dimension four. (Yet a numerical study hints that the claim might also be generalized; see Conjecture 3.54.)

In complete contrast to typical works in topology, where global properties are studied and local characterization is left as general as possible, here we need notations to describe the local view of each vertex of the complex. We needed it for defining the sweep-rule action and stating claims about how it shrinks errors. That led us to the $\theta_{\mathbf{v},\mathbf{S}'}$ notation defined in detail in Definition 3.24. In general, we denote by S the generator set of the Toric, and for a subset of generators $S' \subset S$, we define $\theta_{\mathbf{v},\mathbf{S}'}$ to be the face whose 'bottom-left' corner is the vertex v, and sides of the face correspond to edges on paths obtained by stepping through subsets of S' (for example, if $S' = \{g_1, g_2, g_3\}$ then $\{g_2 g_1 v, g_3 g_2 g_1 v\}$ is an edge of $\theta_{\mathbf{v},\mathbf{S}'}$). Figure 1 shows how the 2D-Toric complex is characterized via $\theta_{\mathbf{v},\mathbf{S}'}$ notation.

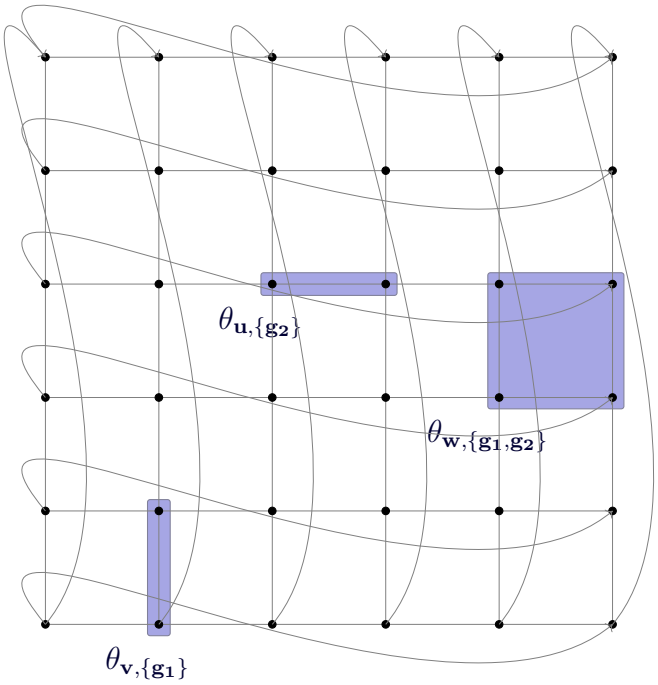

Figure 1: Here we consider a 2D Toric complex. We name the set of generators $S = \{g_1, g_2\}$, where g_1 is the generator that sends us one step in the north direction, and g_2 sends us in the east direction. For vertices v, u, w, we mark in blue three faces that are rooted at them.

The 4D-Toric code is defined as the binary assignment over the 2D-faces (plaquettes), where the Z-stabilizers (parity checks) are set on the edges, namely restrict the Z

product of the faces adjacent to each edge, and the X-stabilizers (phase checks) are set on the 4D-faces (hypercubes). To present the sweep-rule, we need to have a definition of directions. For this section, we will be satisfied with the following hand-waving: For a vertex v, the bits and checks on the first 'quarter' of it (high-dimensional quarter) will be named the positive bits and checks of v.

Definition 2.1 (Sweep Rule (Inforaml))**.** *Let z be a binary assignment on the Toric faces representing 'error'. The sweep rule decoding procedure at a vertex v is defined as follows:*

Measure the positive checks of v and look for an assignment $\tilde{z}$ on the positive bits of v such that the syndromes of the positive checks of v assign the same local syndrome for both z and $\tilde{z}$.

See Definition 3.46 for the formal version. An error correction cycle, or a sweep-cycle, consists of applying the sweep-rule to each vertex and then doing the same in the dual space. Figure 2 provides an example of the result of the sweep rule given a syndrome.

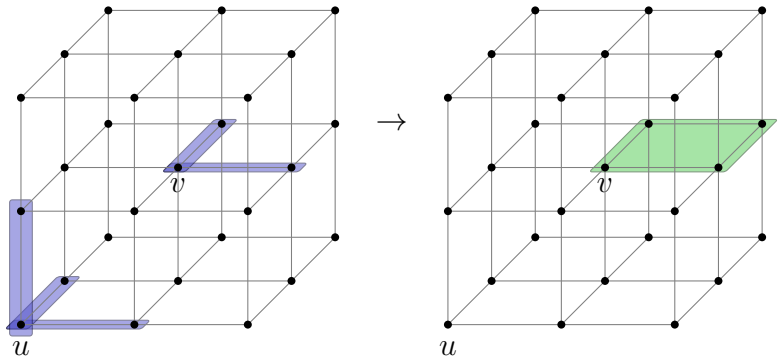

Figure 2: An example of applying the sweep rule within the $(3, 2)$-Toric code. In the $(3, 2)$-Toric code, bits are placed on the squares and Z-stabilizers (parity checks) on the edges. The view is restricted to two vertices, u and v. On the left, the unsatisfied positive checks of u and v are highlighted in blue. On the right, the bits to be flipped are marked in green. Given the syndrome, vertex v flips one of its positive faces, whereas for u, there is no subset of positive faces that, when flipped, decreases the number of local unsatisfied checks. Therefore, u flips no bit.

The way we quantify the 'perimeter' of an error cluster is by considering the (hyper)triangle that bounds the vertices supporting the cluster. The poles of an error cluster are the vertices that lie on the sides of the triangle. We followed [BS88], and by embedding the Toric in the infinite plane, we were able to identify the poles as the max/min points of linear functions which we call *potential walls*. Then, we show that the sweep rule advances only one pole, and hence one side, forward the others. As a consequence, a sweep-cycle shrinks the bounding triangle. The embedding restricts periodicity conditions on the assignments. We encounter this twice: first, an assignment on the Toric's faces that closes a non-trivial cycle translates to an infinite assignment in the infinite plane. In that case, no triangle bounds the error,

which agrees with the fact that a logical error—in that context, an error that changes the logical states—should be 'decoded' to a different codeword. From a technical perspective, it comes into effect as the claims about shrinking assume that a given assignment is a connected finite assignment. The second time we have to deal with the mapping onto the infinite plane is in the calculation of the probability of a given Toom's contour. Since different leaves (faults) of Toom's contour might be associated with the same bit and hence dependent, to bypass this, we took from [Gac21] the claim that states any such graph has a subgraph whose leaves are independent, and their number is at least a constant factor of the original leaves.

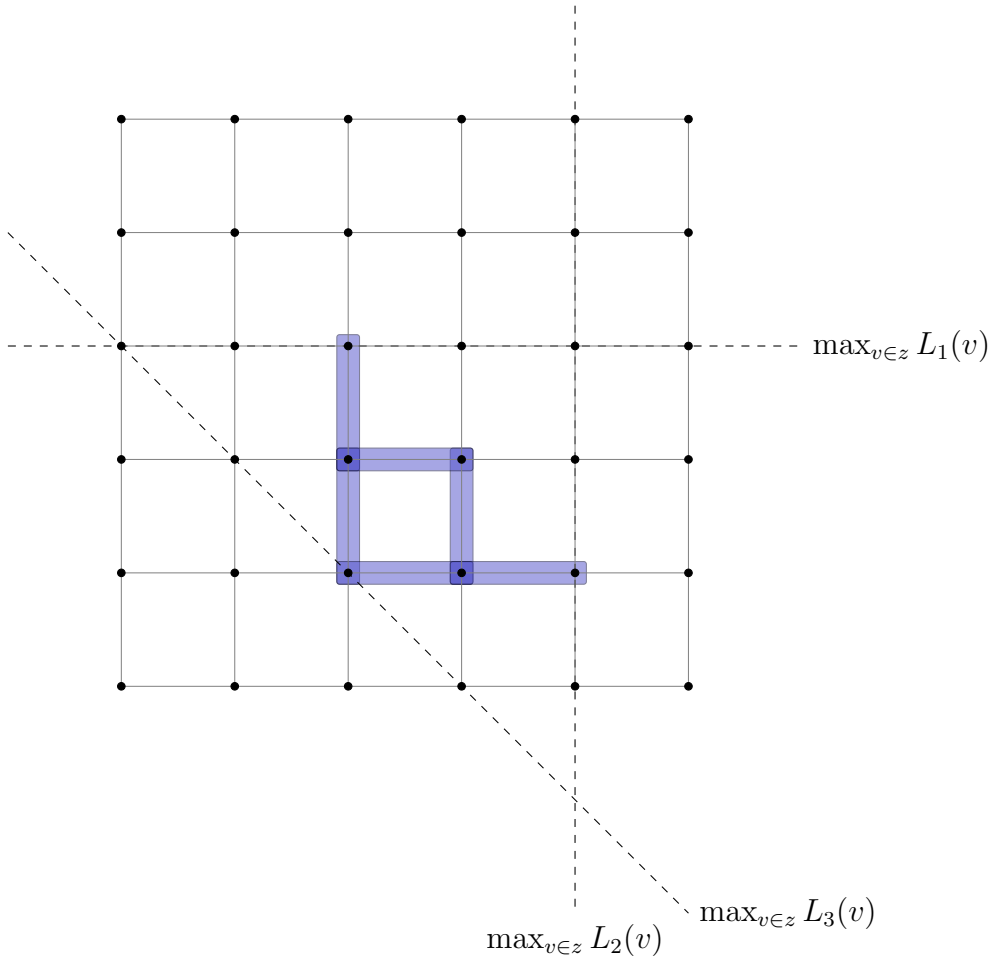

Figure 3: A graphical example of the maximal values of the potential walls. Here we consider the $(2,1)$-Toric code, namely the bits are set on the edges where the Z-stabilizers (parity checks) are set on the vertices. The grid is a finite 6×6 patch of the infinite 2D plane. The edges marked in blue correspond to a bits assignment z. Each of the dashed lines marks the line of vertices whose L_i value converges with the maximal value reached by a vertex in z. Moving into the infinite plane enables us to determine those lines and draw the triangle that bounds z.

See Section 3.4 for embedding into the infinite plane, Section 3.6 for the proof of shrinking, and Section 3.12 for the claim about the existence of a sufficient subgraph.

The Intermediate Theorem. Now that we have the code and decoder, the next difference we have to address is how to classify an error. Unlike the classical regime, flipping a stabilizer's bits doesn't count as a fault. Moreover, the sweep-rule discussed above doesn't shrink stabilizers (as it acts trivially on them). Therefore, we need to redefine what is considered a divergence from the ideal computation. As a result, instead of defining the vertices of Toom's contour graph to be the flipped vertices as in [BS88], we define them to be bits[2] that encounter unsatisfied checks in their local view.

We then face another challenge: if we analyze Toom's contour, which is induced by the single time unit running consisting of flipping all the bits at once, then in the layer following the fault cycle, no bit sees an unsatisfied check. In the language of Toom's contour, as will be defined later, the Toom's contour is empty, and no bounds on the event's probability to occur can be inferred using it. We address this by introducing an analysis framework where all computation, including faults, is imagined to occur sequentially — in steps. Within this framework, we can prove an *intermediate theorem* stating that on the path to a logical error configuration, one must pass through a step with a large cluster of unsatisfied checks. Thus, a sufficient condition for an error to occur in some time unit is the existence of a step with a sufficiently large observed syndrome. In the proof of the theorem, we show that this condition happens with negligible probability. To elaborate on the issue, we first need to present the computation model.

We model a noisy running of a given circuit C as follows: first, we sample space-time locations for the faults and denote the outcome by $\mathcal{E}$. Then, we impose layers of computation alternately, such that the even and odd layers are taken from $\mathcal{E}$ and C according to their order. The combination of the circuit C with a subset of faults $\mathcal{E}$ defines a possible running, and we call it a subjection of C to $\mathcal{E}$ and denote it by $C[\mathcal{E}]$. Furthermore, we showed in Claim 3.23 that for a local-stochastic noise model, unbalancing partition to noisy and logical layers, for example placing a noisy layer after every three logical layers of C, is equivalent to odd-even alternations up to a rescaling of the noise rate (by a constant). We use that since the error correction cycle of the sweep rule takes more than a single layer.

A Tanner graph of an error correction code is a bipartite graph where the left vertices correspond to bits and the right vertices correspond to checks, and there is an edge between a left vertex and a right vertex if they match a bit and a check supported by it. A logical error is an error where an attempt to decode it might lead to a different codeword (different from the original prior to the noise). As stated in [Got14], a sufficient condition for an error to be correctly decodable is that, relative to the Tanner graph of the code, the error is a decomposition of connected components (where two bits are connected if they share a check) such that each of them is comparatively small (at least by a factor of half) to the code distance.

[2] In fact: vertices on the bits' support, see Section 3.8.

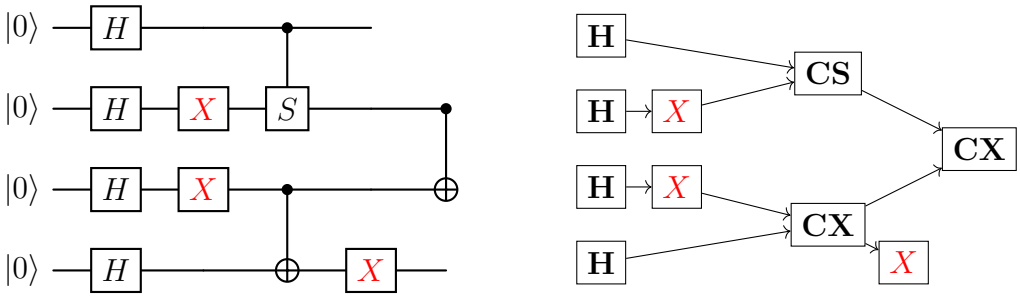

Figure 4: Example of a subjected quantum circuit: the even layers (0, 2, 4) contain the original gates, while the faults are imposed on the odd layers and marked in red.

As mentioned, we are going to analyze the likelihood of getting logical errors by looking back in time and asking what a run that has led to them is supposed to look like. The sweep rule ensures that an error cluster below a certain size shrinks. The problem is that a single fault layer (i.e., a layer whose source is in $\mathcal{E}$) might increase the error cluster above that size. At first glance, one might be tempted to show that the expansion of error clusters beyond a fraction of the code distance happens with negligible probability, but that calculation soon becomes complicated when considering the merging of disjoint clusters. Instead, we split the computation into steps. In each step, only one gate or one fault is applied, and steps are ordered such that they agree with the computation. Figure 4 illustrates a quantum circuit and a graph of steps corresponding to it.

Since the gates and the faults have a bounded fan-in/out[3] In each step, the maximal size of an error cluster can increase by at most a constant factor, with the maximal increase occurring when multiple clusters are merged. Equipped with that insight, we are able to infer the intermediate theorem.

Section 3.2 formalize the computation model and the computation graph presentation (splitting to steps). In particular, see Claim 3.13 and Corollary 3.14 for the formal proof of the intermediate theorem.

Description of the Construction. Similarly to other fault tolerance approaches, we use the concatenation construction in order to initialize zeros and resource states encoded in the 4D-Toric code, as referred to in [Gro19], [Got14]. From there, we assume that all the quantum registers are encoded in the 4D-Toric code, and we name each of the encoded block a quantum register. The resources states are spread across two registers.

Then, after the initialization step, the logical cycles are divided into error correction, which is a single sweep-cycle, and 'logical operation' cycle. For our universal logical gate set, we choose **CX** (controlled NOT), **H**, **S**, and **T** gates. Except for the 'Controlled NOT' gate, which is realized by transversal wire-wise physical 'Controlled

[3]The faults are bounded-fan because we can assume that in the time units representation they are Pauli products and therefore in the steps presentation they are decomposed into single qubit Pauli matrices.

NOT' gates, all the others are implemented via gate teleportation [ZLC00].

The general realization by gate teleportation is done as follows: first, we XOR the target state into a resource state via **CX**, measure the input state and the first register of the resource state with a Bell measurement, the input register in the X basis, and the first register of the resource state in the Z basis. Then, on the classical outcomes, run a classical decoding procedure to extract the logical codeword. Meanwhile, on the second register of the resource, which is intended to contain the result of the action, perform a sweep cycle. At the end of the decoding, depending on the result, we apply either a transversal Pauli operator or the **S** gate (again by teleportation) depending on the original gate that was asked to simulate. The gate teleportation lasts for multiple logical cycles, depending on the depth required by the classical decoding procedure.

We emphasize that for measuring the input register in the X-basis, we apply the Hadamard through its fold-transversal realization [BB24], so the measurement is done in constant depth. However, the reason we don't implement the Hadamard by fold-transversal from the beginning is that we have failed to provide proof that restricts the expansion of the bounded-triangle[4] by the fold-transversal realization. Since it guarantees that after the measurement the register will not observe more faults, we can tolerate a one-time blowup in the size of the error and still be able to decode the logical state in the register correctly.

In Section 3.3, we discuss in detail the structures of the logical codeword of the 4D-Toric code and the realization of its logical gates.

Toom's Contours. We construct the Toom's contours as the follow graph $\mathcal{H} := (V_{\mathcal{H}}, E_{\mathcal{H}})$:

1. If at time t, a vertex v, on the one the Toric complexes[5] corresponding to one of the quantum registers, sees an unsatisfied check on it's local view, then add (v, t) to the vertices.

2. If at time t, vertex v sees an unsatisfied check, and at time $t+1$ its neighbor u sees an unsatisfied check, then we think of the situation as 'v 'leads' to u' and connect (v, t) and $(u, t+1)$ with an 'arrow'. The vertices v and u might belong to different registers (which matches the case that at time t we apply an operator on both registers). For convenience, we don't require u at time t to see only satisfied checks.

3. As in the classical case, connect any two vertices in $V_{\mathcal{H}}$ that are connected to a third by 'arrows' with a 'fork'.

Later, we incorporate this idea into the formation of Toom's counter, i.e., $\mathcal{H}$-graphs, by gluing the layers corresponding to the first $t-1$ layers with a final layer that matches a partial execution of the tth time unit. The layer encodes the situation of the running up to a certain step, in which the syndrome is large and therefore derives a non-trivial Toom's contour.

[4]The permutation used in the Hadamard realizations mirrors the errors

[5]Actually the infinite grid.

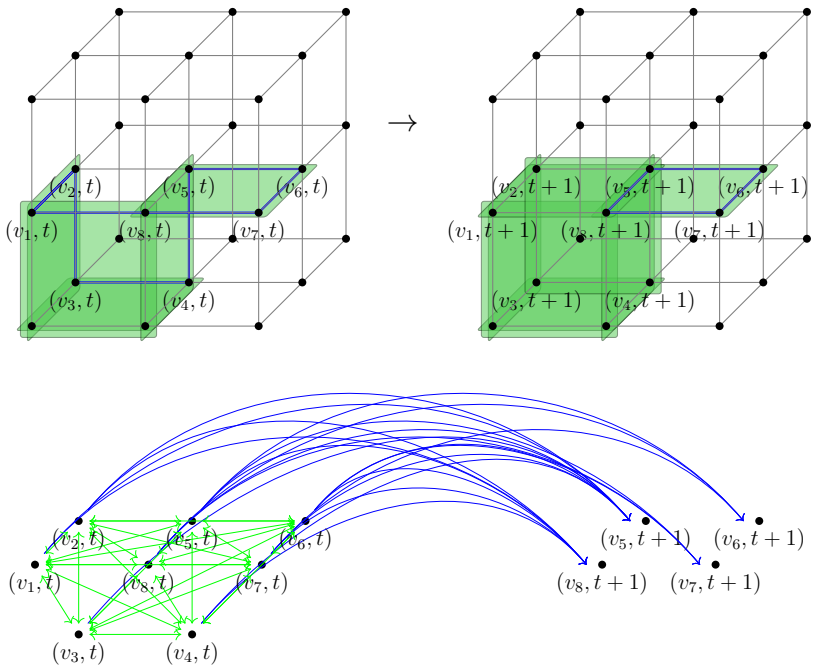

Figure 5: Above: The application of a sweep-cycle, where the green faces represent the bits assignment z, and the blue edges correspond to unsatisfied checks ∂z. The vertices in ∂z are labeled as $v_1, v_2, \ldots, v_8$. Below: The Toom's contour derived from the single computation layer that matches the cycle. Arrows are colored in blue, while forks are colored in green. Note that $(v_1, t+1)$ doesn't belong to the Toom's contour (since v_1 doesn't belong to the syndrome at time $t+1$).

3 The Proof in Detail

3.1 Notations

We follow the standard notations commonly used in the literature. For an integer n, we denote by $[n]$ the set of integers $\{1, 2, \ldots, n\}$. To present a countable set, which might or might not be finite, where the number of elements does not matter, we use curly brackets with a subscript denoting the first index of the set. For example, $\{u_i\}_{i=1}$ represents the set $u_1, u_2, \ldots$; For finite sets, we might add a superscript to present the size of the set. We use a colon inside curly brackets to specify a set of elements satisfying a condition, i.e., for sets A, B, the notation $\{a : a \in A, a \in B\}$ equals $A \cap B$. Curly brackets without subscripts or conditions refer to singletons. For example, for a given integer i, the set $\{u_i\}$ is the set that contains only the element u_i. For an integer i, we denote by 1_i the vector consisting of one at the ith coordinate and zero elsewhere.

When not otherwise mentioned, we consider quantum states over qubits, each lying in a two-dimensional Hilbert space $\mathcal{H}_2$, and denote by $\mathcal{H}_{2^n}$ the Hilbert space of an n-qubit system. Pure quantum states and their dual presentations are denoted by $|\psi\rangle, \langle\psi|$, and in the matrix representation, we use $|\psi\rangle\langle\psi|$ to denote its density

matrix. We will use ρ to denote a mixed state, whose density matrix is a trace one PSD matrix.

Below, a semi proof that in CSS code partial correction $Z^e \to Z^{\hat{e}}$ doesn't add bit flip errors. It might adds a global phase. I should enter it in a section about CSS coding.

$$\begin{aligned}
& X^f Z^e \left| w + C_z^\perp \right\rangle \\
=& X^f Z^e X^w \left| C_z^\perp \right\rangle \\
\text{apply } H^{\otimes n} \mapsto& Z^f X^e Z^w \left| C_z \right\rangle \\
=& (-1)^{e\cdot w} Z^f Z^w X^e \left| C_z \right\rangle \\
\text{partial correction } X^{e+\hat{e}} \mapsto& (-1)^{(e+\hat{e})(f+w)} (-1)^{e\cdot w} Z^f Z^w X^{\hat{e}} \left| C_z \right\rangle \\
\text{apply } H^{\otimes n} \mapsto& (-1)^{(e+\hat{e})(f+w)} (-1)^{e\cdot w} X^f X^w Z^{\hat{e}} \left| C_z^\perp \right\rangle \\
=& (-1)^{(e+\hat{e})(f+w)} (-1)^{e\cdot w} (-1)^{\hat{e}\cdot w} X^f Z^{\hat{e}} X^w \left| C_z^\perp \right\rangle \\
=& (-1)^{(e+\hat{e})f} X^f Z^{\hat{e}} \left| w + C_z^\perp \right\rangle
\end{aligned}$$

3.2 Computation Model

Informally, a quantum circuit is a placement of a collection of quantum gates, taken from a finite set, at different space-time locations. Each such location records when the gate is applied and on which qubits it acts. Formally:

Definition 3.1 (Quantum Circuit)**.** *A* space-time location *is a pair $l = (t, S)$, where t is a time coordinate and $S = (s_1, \ldots, s_\Delta)$ is an ordered tuple of qubit indices. For a finite collection of quantum gates*

$$\mathcal{G} \subset \left\{ g \colon L(\mathcal{H}_2^{\otimes \Delta}) \to L(\mathcal{H}_2^{\otimes \Delta}) \right\},$$

and integers $w, d \in \mathbb{N}$, a quantum circuit C of depth d and width w is a collection of tuples

$$C = \{u_i = (g_i, l_i)\}_i,$$

where each $g_i \in \mathcal{G}$ and each location $l_i = (l_{i,1}, l_{i,2})$ belongs to $\mathbb{Z}_d \times \mathbb{Z}_w^\Delta$. We call the tuples u_i the gates of C. They must satisfy the following compatibility condition: if two gates have the same time coordinate, then their supports are disjoint. Namely, for $u_i = (g_i, l_i), u_j = (g_j, l_j) \in C$, the equality $l_{i,1} = l_{j,1}$ implies that the sets of qubit indices appearing in $l_{i,2}$ and $l_{j,2}$ are disjoint. We denote by

$$\mathrm{Loc}(C) := \{l_i : u_i = (g_i, l_i) \in C\}$$

the set of locations occupied by the gates of C. We define the t-layer of C to be its restriction to gates at time t:

$$C|_t := \{u_i = (g_i, l_i) \colon u_i \in C \text{ and } l_{i,1} = t\}.$$

To define the action of a quantum circuit C on a given mixed state, we first associate to C a directed acyclic graph.

Definition 3.2 (Associated Computation DAG)**.** *Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w. We define the associated computation DAG of C, denoted by π_C, to be the directed acyclic graph $\pi_C = (\mathcal{U}, E)$ as follows:*

1. *The vertex set is the gate set of C, namely $\mathcal{U} = \{u_i\}_i$.*
2. *For $u_i = (g_i, l_i)$ and $u_j = (g_j, l_j)$, there is a directed edge $(u_i, u_j) \in E$ if and only if j is one of the minimizers of*

$$\big\{l_{k,1} : (g_k, l_k) \in \mathcal{U},\ l_{k,1} > l_{i,1},\ l_{k,2} \cap l_{i,2} \neq \emptyset\big\}.$$

Equivalently, u_j is one of the earliest later gates whose support intersects the support of u_i. There may be more than one such minimizer, so the out-degree may exceed one.

Suppose that $J = (j_1, j_2, \ldots, j_k)$ is an ordering of the gate indices that agrees with a topological order induced by π_C. Then we define the associated partial circuits $C_J = \{C_{J,i}\}_{i=1}^k$ by

1. *$C_{J,1} = \{u_{j_1}\}$.*
2. *For $1 \leq i < k$, set*

$$C_{J,i+1} = C_{J,i} \cup \{u_{j_{i+1}}\}.$$

We call $C_{J,i}$ the partial computation circuit up to step i.

Definition 3.3. *Let ρ be a density matrix over w qubits, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with depth d and width w. For any density matrix σ, quantum gate $g \in \mathcal{G}$, and location l, the application of the gate-location tuple (g, l) on σ, denoted by $(g, l) \circ \sigma$, is the application of g on σ where wires are ordered such that the first qubit in the input to g is $l_{2,1}$, the second is $l_{2,2}$, and so on. The result of the application of C on ρ, denoted by $C \circ \rho$, is given by iterative applications of the gates $\{u_i\}_{i=1}$ on ρ, according to a topological order induced by π_C.*

We say that $\rho_1, \rho_2, \ldots, \rho_k$ is a computation prefix if $\rho_1 = \rho$ and $\rho_{i+1} = u'_i \circ \rho_i$, where $\{u'_i\}_{i=1}$ is a prefix of a topological ordering of $\{u_i\}_{i=1}$. If $\{u'_i\}_{i=1}$ contains all the gates in $\{u_i\}_{i=1}$, then we call $\rho_1, \rho_2, \ldots, \rho_k$ a running.

Definition 3.4 (C subjected to $\mathcal{E}$ every m computational layers)**.** *Let $C = \{u_i\}_i$ be a quantum circuit with depth d and width w, let $m \in \mathbb{N}$, and set*

$$d_m := \left\lceil \frac{d}{m} \right\rceil.$$

Let $\mathcal{L} \subset [d_m] \times [w]$ be a set of locations, and let $\mathcal{E}$ be a quantum circuit, such $Loc(\mathcal{E}) = \mathcal{L}$. We name $\mathcal{E}$ subsets of faults, and we name the layers of $\mathcal{E}$ fault layers. The circuit C

subjected to $\mathcal{E}$ every m computational layers, denoted by $C[\mathcal{E}]_m$, is obtained by keeping the order of the computational layers of C and inserting one fault layer after every block of m consecutive computational layers. Thus the first fault layer is inserted after the computational layers $1, \ldots, m$, the second is inserted after the computational layers $m+1, \ldots, 2m$, and so on. Concretely, each computational gate $u_i = (g_i, (t_i, S_i))$ of C is moved to the time coordinate

$$t_i + \left\lfloor \frac{t_i - 1}{m} \right\rfloor,$$

while each fault gate $(g, (t, S)) \in \mathcal{E}$ is placed at time coordinate

$$t(m+1).$$

Equivalently, for each time index $t \in [d_m]$, the computational layers with original times

$$(t-1)m + 1, \ldots, \min\{tm, d\}$$

appear consecutively and are followed by the fault layer at time $b(m+1)$. When $m = 1$, the computational gates occupy the odd time coordinates and the fault layers occupy the even time coordinates. When the parameter is omitted, we mean $m = 1$. We denote the associated computation DAG of $C[\mathcal{E}]_m$ by $\pi[C, \mathcal{E}]$.

Remark 3.5. *Formally, a fault circuit is just a particular quantum circuit whose gates are interpreted as faults rather than as computational operations. Thus the distinction between a quantum circuit and a fault circuit is semantic rather than structural: both are circuits in the sense of Definition 3.1, but they play different roles in the analysis.*

Example 3.6. *When $m = 3$, the computational layers are sent to the times*

$$1, 2, 3, 5, 6, 7, 9, 10, 11, \ldots,$$

while the fault layers are sent to the times

$$4, 8, 12, \ldots.$$

The chapter uses two different notions of propagation. They are related, but they serve different analytical purposes. The first notion does not rewrite the circuit and does not change the time coordinates. Instead, it propagates part of the error inside a fixed running of the computation and asks what the accumulated effect looks like at a chosen step τ. In particular, this notion applies not only to a single fault, but also to a collection of faults propagated one after another according to the computation DAG. This is the right framework when one wants to speak about the size of the syndrome or of the deviation *at an actual intermediate time* of the computation.

This distinction is essential for statements such as the intermediate value theorem proved below. For example, a codeword may absorb noise during one time unit and be mapped to another codeword. If one only commutes whole fault layers forward,

then one reorganizes the circuit globally and may lose the ability to point to the time at which a large syndrome first appears. The first notion of propagation avoids this problem: by propagating only part of the error while keeping the underlying time axis fixed, it lets us identify intermediate steps where a medium-size or large deviation must already be visible.

The second notion is more global. There we swap entire layers and then update the time coordinates accordingly. Its role is not to locate when a syndrome becomes large, but rather to compare different time-unfoldings of the same noisy circuit and to coarse-grain the noise into sparser effective layers. In particular, once the theorem proved using this second notion is established, any constant-depth collection of gates may be treated as a single time step for the purpose of the noise analysis: up to the corresponding effective rescaling of the noise rate, we may ignore the faults that occur during the internal application of those gates and replace them by an effective fault layer acting only before or after that constant-depth subcircuit. Moreover, this effective rescaling is correct with respect to local stochastic noise channels: a local stochastic noise channel is mapped to another local stochastic noise channel, with a rescaled noise rate.

For readability, we therefore separate the discussion into two subsections. The first subsection develops propagation inside a fixed running and is tailored to the deviation-cluster bounds, while the second develops layer propagation and is tailored to the coarse-graining argument.

3.2.1 Local Propagation Along a Running

Definition 3.7 (Fault propagation to a step τ)**.** *Let $C = \{u_i\}_i$ be a quantum circuit, let $J = (j_1, \dots, j_k)$ be an order agreeing with π_C, and let $u_i \in C$. Suppose that $u_i = u_{j_l}$ in this order. If $l > \tau$, we define the propagation of u_i to step τ to be the identity operator.*

Otherwise, define operators $X_0, X_1, \dots, X_{\tau-l}$ recursively by setting $X_0 = u_{j_l}$ and requiring that for every $1 \le s \le \tau - l$,

$$X_{s-1} u_{j_{l+s}} = u_{j_{l+s}} X_s.$$

We then define the propagation of u_i to step τ *to be the operator*

$$X := X_{\tau-l}.$$

Now let $C[\mathcal{E}]$ be a noisy circuit, let J be an order agreeing with $\pi_{C[\mathcal{E}]}$, and let τ be a step in that order. For each fault gate $e \in \mathcal{E}$ that occurs no later than step τ, denote by X_e^τ its propagation to step τ. We define the propagated error at step τ by multiplying these propagated faults in reverse topological order:

$$X^\tau := \prod_{e \preceq \tau} X_e^\tau.$$

Definition 3.8 (Deviation)**.** *For a quantum circuit C, a set of faults $\mathcal{E}$, and step τ, let X^τ be the propagated error at step τ. We define the deviation of $C[\mathcal{E}]$ from C at step τ by the set of indices on which X^τ acts non-trivially.*

Definition 3.9. *Using the same notations as in Definition 3.8, for a set of unitary operators $\mathcal{S} = \{s_i \colon \mathcal{H}_2^w \to \mathcal{H}_2^w\}_{i=1}$, we define the propagated error up to stabilizers in $\mathcal{S}$ as $X^\tau[S] := \min_{s\in\mathcal{S}}\{w(sX)\}$. Then, we define the deviation of $C[\mathcal{E}]$ from C at step τ up to stabilizers in $\mathcal{S}$ to be the subset of coordinates on which $X^\tau[S]$ acts non-trivially.*

The following definitions extend the notion of a Tanner graph to the setting of quantum circuits.

Definition 3.10 (Tanner graph of a register)**.** *Let C be a quantum circuit and let R be a register, namely a subset of qubits of C. The Tanner graph of R at step τ, denoted by $\mathcal{T}_{R,\tau}$, is defined as follows:*

1. *If at step τ the register R is encoded by a code $\mathcal{C}$, then $\mathcal{T}_{R,\tau}$ is the Tanner graph of $\mathcal{C}$.*
2. *Otherwise, $\mathcal{T}_{R,\tau}$ is the empty graph.*

When the step is clear from the context, we omit the subscript τ.

Definition 3.11 (Generalized Tanner graph of a circuit)**.** *Let C be a quantum circuit. For registers A and B in C, define the generalized Tanner graph of their union by*

$$\mathcal{T}_{A\cup B,\tau} := \mathcal{T}_{A,\tau} \cup \mathcal{T}_{B,\tau}.$$

More generally, for a partition of the qubits into registers $\{R_i\}_i$, the generalized Tanner graph of C at step τ is

$$\mathcal{T}_{C,\{R_i\},\tau} := \bigcup_i \mathcal{T}_{R_i,\tau}.$$

When the partition and the step are clear, we abbreviate this notation to $\mathcal{T}_C$.

Definition 3.12 (Clustered deviation)**.** *Let C be a quantum circuit, let R be a register of C, and let $\mathcal{E}$ be a fault set. Let $\mathcal{T}_{R,\tau}$ be the Tanner graph of R at step τ, and let $\mathcal{D}_\tau$ be the deviation of $C[\mathcal{E}]$ from C on R at step τ. Denote by $\mathcal{T}_{R,\tau}|_{\mathcal{D}_\tau}$ the subgraph induced on the left vertices corresponding to the qubits in $\mathcal{D}_\tau$.*

We say that two left vertices are connected if they share a common right neighbor. The deviation clusters at step τ are the connected components of $\mathcal{T}_{R,\tau}|_{\mathcal{D}_\tau}$. The size of a deviation cluster is the number of right vertices it contains.

Claim 3.13. *Let C be a quantum circuit whose gates act on at most Δ qubits. Let R be a quantum register encoded by a quantum error-correcting code whose Tanner graph has left degree at most Δ_1 and right degree at most Δ_2. Fix a fault set $\mathcal{E}$ and an order J agreeing with $\pi_{C[\mathcal{E}]}$. For a step $\tau \in J$, let $Y_1^{(\tau)}, Y_2^{(\tau)}, \ldots$ denote the deviation clusters at step τ.*

Suppose that at step τ_1 there is a deviation cluster $Y_i^{(\tau_1)}$. Then there exist deviation clusters Z and Z' at step $\tau_1 - 1$ such that

$$Z \cap Y_i^{(\tau_1)} \neq \emptyset, \qquad |Z| \geq \frac{1}{\Delta\Delta_1\Delta_2}\,|Y_i^{(\tau_1)}|,$$

and

$$Z' \cap Y_i^{(\tau_1)} \neq \emptyset, \qquad |Z'| \leq \Delta\Delta_1\Delta_2\,|Y_i^{(\tau_1)}|.$$

Proof. Let u be the gate applied at step τ_1. Passing from step $\tau_1 - 1$ to step τ_1 can modify the deviation only on qubits touched by u. Since u acts on at most Δ qubits, and each such qubit is adjacent in the Tanner graph to at most Δ_1 checks, each of which meets at most Δ_2 qubits, the gate u can interact with vertices belonging to at most

$$\Delta\Delta_1\Delta_2$$

deviation clusters from step $\tau_1 - 1$.

Let $Y_1, \ldots, Y_m$, with $m \leq \Delta\Delta_1\Delta_2$, be those deviation clusters at step $\tau_1 - 1$ that meet the neighborhood affected by u and whose union contains $Y_i^{(\tau_1)}$. Then

$$|Y_i^{(\tau_1)}| \leq \sum_{j=1}^{m} |Y_j| \leq m \max_{1\leq j\leq m} |Y_j| \leq \Delta\Delta_1\Delta_2 \max_{1\leq j\leq m} |Y_j|.$$

Hence one of these clusters, call it Z, satisfies

$$|Z| \geq \frac{1}{\Delta\Delta_1\Delta_2} |Y_i^{(\tau_1)}|.$$

By construction, $Z \cap Y_i^{(\tau_1)} \neq \emptyset$.

For the upper bound, apply the same argument to the inverse gate $u^\dagger$, which also acts on at most Δ qubits. Viewing the transition from step τ_1 back to step $\tau_1 - 1$ as the application of $u^\dagger$, we conclude that some deviation cluster Z' at step $\tau_1 - 1$ intersecting $Y_i^{(\tau_1)}$ satisfies

$$|Z'| \leq \Delta\Delta_1\Delta_2 \, |Y_i^{(\tau_1)}|.$$

This proves both bounds. □

Corollary 3.14 (Intermediate Value Theorem)**.** *Let $\alpha = 1/(\Delta\Delta_1\Delta_2)$, and let x be an integer. Assume that at the initial step τ_o, there is no deviation cluster of size more than αx, and that at a step $\tau_1 \in J$, such that $\tau_1 > \tau_o$, there is a deviation cluster $Y^{(\tau_1)}$ of size $|Y^{(\tau_1)}| > x$. Then there is a step $\tau_\star \in J$ such that $\tau_o < \tau_\star < \tau_1$ for which there is a deviation cluster Z at step $\tau_\star$, satisfying that the size of Z is in the range $(\alpha x, x)$.*

Proof. Assume for contradiction that for every step τ with $\tau_o < \tau < \tau_1$, all deviation clusters have size at most αx. In particular, this holds at step $\tau_1 - 1$. On the other hand, since $|Y^{(\tau_1)}| > x$, Claim 3.13 implies the existence of a deviation cluster at step $\tau_1 - 1$ of size at least

$$\frac{1}{\Delta\Delta_1\Delta_2}|Y^{(\tau_1)}| = \alpha|Y^{(\tau_1)}| > \alpha x,$$

which is a contradiction. Therefore there must exist an intermediate step $\tau_\star$ with $\tau_o < \tau_\star < \tau_1$ at which some deviation cluster has size in the interval $(\alpha x, x)$. □

Claim 3.15. *Let $\alpha = (\Delta\Delta_1\Delta_2)^{-1}$. Let C be a quantum circuit with order J and depth d, and let $\mathcal{E} \sim \mathcal{N}_\mathcal{E}$, so that $C[\mathcal{E}]$ is a random circuit whose gates act on at most Δ qubits. Fix a time $t \leq 2d$ and an integer $x \in \mathbb{N}$. Let A be the event that by time t there exists a deviation cluster of size greater than x. For each step $\tau \in J$, let B_τ be the event that step τ is the first step at which there exists a deviation cluster of size in the interval $(\alpha x, x)$. Then*

$$\mathbf{Pr}\big[A\big] \leq 2wd \max_{\tau\in[2wd]} \mathbf{Pr}\big[B_\tau\big].$$

Proof. By Corollary 3.14, whenever the event A occurs there exists a step $\tau \in J$ with $\tau \leq 2wt$ at which there is a deviation cluster of size in the interval $(\alpha x, x)$. Hence

$$A \subset \bigcup_{\tau\in[2wt]} B_\tau \subset \bigcup_{\tau\in[2wd]} B_\tau.$$

The union bound now gives

$$\mathbf{Pr}\big[A\big] \leq \mathbf{Pr}\big[\bigcup_{\tau\in[2wd]} B_\tau\big] \leq \sum_{\tau\in[2wd]} \mathbf{Pr}\big[B_\tau\big] \leq 2wd \max_{\tau\in[2wd]} \mathbf{Pr}\big[B_\tau\big].$$

□

3.2.2 Layer Propagation and Time Coarse-Graining

We now switch from propagation along a fixed running to propagation at the level of entire time slices. This second notion is intentionally stronger and less local. In the previous subsection, the purpose was to say: given a fault that already occurred, what is its effect at a later step of the same computation? Here the purpose is different: we want to reorganize the circuit itself, by commuting fault layers forward and grouping them into coarser effective layers. This is the mechanism behind the later renormalization claim for local stochastic noise.

In particular, local propagation keeps the ambient circuit fixed and changes the *description of the error*. Layer propagation, by contrast, changes the *presentation of the circuit* while preserving the implemented channel. This is why the two notions should not be merged into one definition: one is adapted to tracking deviations, while the other is adapted to comparing circuits related by time-unfolding.

Definition 3.16 (Layer propagation)**.** *Let $C = \{u_i\}_{i=1}$ be a quantum circuit. For each time coordinate t, write $U_t := C|_t$. The family of non-empty t-cuts $\{U_t\}_t$ is called the layer decomposition of C.*

Let $t < t'$ be such that U_t and $U_{t'}$ are non-empty and $U_s = \emptyset$ for every $t < s < t'$. The forward propagation of the layer U_t across the next layer $U_{t'}$ is the operation that replaces the pair $(U_t, U_{t'})$ by a pair $(U_{t'}, V_{t\to t'})$ satisfying

$$\prod_{u\in U_t} u \prod_{v\in U_{t'}} v = \prod_{v\in U_{t'}} v \prod_{w\in V_{t\to t'}} w.$$

The layer $V_{t\to t'}$ is called the propagation expression of U_t across $U_{t'}$.

More generally, if $t < t'$, the forward propagation of U_t to time t' is obtained by iterating the above operation through the successive non-empty cuts between times t and t'. The resulting propagated layer is denoted by $V_{t\to t'}$ and is again called the propagation expression of U_t to time t'. The resulting circuit has layer decomposition

$$\ldots, U_{t-1}, U_t, U_{t+1}, \ldots, U_{t'}, U_{t'+1}, \ldots \mapsto \ldots, U_{t-1}, U_{t+1}, \ldots, U_{t'}, V_{t\to t'}, U_{t'+1}, \ldots.$$

After propagation we update the time coordinates so that they agree with the new layer in which each gate is applied.

Claim 3.17. *Assume that every gate in C acts on at most Δ qubits. Then every circuit obtained from C by forward layer propagation also consists of gates acting on at most Δ qubits.*

Proof. Propagating a gate $u \in U_i$ across a layer U_{i+1} conjugates u by the product of the gates in U_{i+1} that intersect its support. Since the gates inside U_{i+1} are pairwise disjoint, each qubit in the support of u participates in at most one gate from U_{i+1}. Therefore the support of the propagated image of u is contained in the union of the supports of at most Δ gates, each of arity at most Δ, but only one such gate can be associated with each qubit of u. Consequently the propagated image still acts on at most Δ wires. Applying this argument repeatedly proves the claim. □

Definition 3.18 ($C \sim_P C'$)**.** *For quantum circuits C and C', we write $C \sim_P C'$ if one can be obtained from the other by a finite sequence of layer propagations.*

By construction, $\sim_P$ preserves the implemented channel. Indeed, if $C \sim_P C'$ and ρ is any mixed state on the input register, then

$$C \circ \rho = C' \circ \rho.$$

Definition 3.19 (Noise channel)**.** *A noise channel $\mathcal{N}_\mathcal{E}$ is a distribution over sets of fault locations.*

Definition 3.20 (Local stochastic noise channel)**.** *Let $p \in (0, 1)$. We say that the noise channel $\mathcal{N}_\mathcal{E}$ is local stochastic with rate p if for every finite set of locations $S \subset \mathbb{N} \times \mathbb{N}$,*

$$\mathbf{Pr}_{\mathcal{E}\sim\mathcal{N}_\mathcal{E}}\big[S \subset \mathrm{Loc}(\mathcal{E})\big] \le p^{|S|}.$$

Given a circuit C and a random fault set $\mathcal{E} \sim \mathcal{N}_\mathcal{E}$, the subjected circuit $C[\mathcal{E}]$ is itself a random variable.

Definition 3.21 (Witness family in the backward light cone)**.** *Let W be a propagation expression obtained from a block of c layers of a noisy circuit after propagating all even fault layers to the end of the block. For a subset $S \subset \mathrm{Loc}(W)$, we define the witness family of S, denoted by $\mathcal{W}(S)$, to be the collection of all minimal sets*

$$S' \subset \Lambda(S)$$

such that there exists a fault pattern containing S' whose propagation produces all the faults at the locations in S. Here minimality is with respect to inclusion.

Claim 3.22. *Let $c \geq 2$ be even, let C be a depth-$c/2$ circuit whose gates have arity at most Δ, and let $C[\mathcal{E}]$ be the circuit obtained by inserting the faults of $\mathcal{E}$ between the computational layers. Write the layer decomposition of $C[\mathcal{E}]$ as*

$$U_0, U_1, \ldots, U_c,$$

where the even layers are fault layers and the odd layers are computational layers. Propagate the fault layers $U_2, U_4, \ldots, U_{c-2}$ forward beyond U_{c-1}, in reverse order, and denote by $V^{(1)}$ the resulting propagation expression obtained by grouping together those propagated layers with the terminal fault layer U_c. Then for every $S \subset \mathrm{Loc}(V^{(1)})$ the witness family $\mathcal{W}(S)$ is non-empty, and every element $S' \in \mathcal{W}(S)$ satisfies

$$S' \subset \Lambda(S) \cap \bigcup_{k=0}^{c/2} U_{2k}$$

and

$$|S'| \geq \frac{|S|}{\Delta^c}.$$

Consequently,

$$\{S \subset \mathrm{Loc}(V^{(1)})\} \subset \bigcup_{S' \in \mathcal{W}(S)} \{S' \subset \mathrm{Loc}(\mathcal{E})\}.$$

Proof. Suppose that the event $\{S \subset \mathrm{Loc}(V^{(1)})\}$ occurs. Then there exists at least one set of original fault locations whose propagation produces all the faults in S. Among all such sets choose one that is minimal with respect to inclusion, and denote it by S'. By construction, $S' \in \mathcal{W}(S)$, so the witness family is non-empty.

Since every element of S is obtained by propagating some original fault through at most c layers, and propagation never leaves the backward light cone of the produced qubits, every location in S' lies in $\Lambda(S) \cap \bigcup_{k=0}^{c/2} U_{2k}$.

It remains to bound the size of S'. A single fault location can influence at most Δ^c qubits in the final layer of the block, because at each of the at most c propagation steps the support can branch to at most Δ qubits. Therefore any set of original faults capable of generating all elements of S must contain at least $|S|/\Delta^c$ fault locations. In particular, this holds for the minimal producing set S', so $|S'| \geq |S|/\Delta^c$.

Finally, whenever $S \subset \mathrm{Loc}(V^{(1)})$ occurs, at least one minimal producing set $S' \in \mathcal{W}(S)$ is fully contained in the realized fault set $\mathrm{Loc}(\mathcal{E})$. This proves the event inclusion. □

Claim 3.23. *Let $\mathcal{N}_{\mathcal{E}}$ be a local stochastic noise channel with rate p, and let C be a quantum circuit whose gates have arity at most Δ. Fix an even integer c. Then there exists a local stochastic noise channel $\mathcal{N}_{\mathcal{E}_q}$ with rate*

$$q := \left(2^{H(\Delta^{-c})\Delta^c} p\right)^{\Delta^{-c}},$$

such that

$$C[\mathcal{E}]_1 \sim_P C[\mathcal{E}_q]_{c/2}.$$

In particular, up to a constant depending only on Δ and c, the rate q behaves like $p^{\Delta^{-c}}$.

Proof. Partition the subjected circuit $C[\mathcal{E}]_1$ into consecutive blocks of c layers. Inside each block, propagate all internal even fault layers to the end of the block, exactly as in Claim 3.22. Doing this block by block yields a new circuit C' in which each block of $c/2$ computational layers is followed by a single coarse-grained fault layer. Therefore $C' = C[\mathcal{E}']_{c/2}$ for a suitable random fault set $\mathcal{E}'$, and by construction

$$C[\mathcal{E}]_1 \sim_P C[\mathcal{E}']_{c/2}.$$

It remains to prove that $\mathcal{E}'$ is local stochastic. Fix one coarse-grained propagation expression $V^{(i)}$ and a subset $S \subset \mathrm{Loc}(V^{(i)})$. By Claim 3.22, the event $\{S \subset \mathrm{Loc}(V^{(i)})\}$ implies that at least one witness set $S' \in \mathcal{W}(S)$ is fully contained in the original fault set $\mathrm{Loc}(\mathcal{E})$, and every such witness satisfies $|S'| \geq |S|/\Delta^c$. Hence

$$\begin{aligned}\mathbf{Pr}\big[S \subset \mathrm{Loc}(V^{(i)})\big] &\leq \mathbf{Pr}\Big[\bigcup_{S' \in \mathcal{W}(S)} \{S' \subset \mathrm{Loc}(\mathcal{E})\}\Big] \\ &\leq \sum_{S' \in \mathcal{W}(S)} \mathbf{Pr}\big[S' \subset \mathrm{Loc}(\mathcal{E})\big].\end{aligned}$$

The channel $\mathcal{N}_{\mathcal{E}}$ is local stochastic with rate p, so each term is at most $p^{|S'|}$. Moreover, the backward light cone of each effective fault has size bounded by a constant depending only on Δ and c, so the number of candidate witnesses of size ℓ is at most

$$\binom{\Delta^c |S|}{\ell}.$$

Choosing $\ell = \lceil |S|/\Delta^c \rceil$ and using the standard entropy bound

$$\binom{\Delta^c |S|}{|S|/\Delta^c} \leq 2^{H(\Delta^{-2c})\Delta^c |S|},$$

we obtain

$$\mathbf{Pr}\big[S \subset \mathrm{Loc}(V^{(i)})\big] \leq \Big(2^{H(\Delta^{-2c})\Delta^c} p\Big)^{|S|/\Delta^c} = q^{|S|}.$$

Since this bound is uniform over all subsets S and all coarse-grained layers, the induced random set $\mathcal{E}'$ is local stochastic with rate q. Setting $\mathcal{E}_q = \mathcal{E}'$ completes the proof. □

3.3 The (d, d')-Dimensional Toric Code

Definition 3.24 (d-Dimensional Toric Complex)**.** *For integers d and n, denote the group resulting from the d-directed power of the cyclic group of order n by $\mathcal{G}_n^d = \mathbb{Z}_n^{\times d}$, and denote the standard generating set of $\mathcal{G}_n^d$ by $S = \{1_i \colon i \in [d]\}$. We define the d-dimensional toric complex to be the hypergraph obtained from the Cayley graph $Cay(\mathcal{G}_n^d, S)$ by taking its vertices as the 0-faces and for any $i \in [d-1]$ we define the i-faces as follow: First, for a vertex $v \in \mathcal{G}_n^d$ and for a subset of generators $S' \subset S$ of size $|S'| = i$, the i-face rooted at v and spanned by S', denoted by $\theta_{v,S'}$, is:*

$$\theta_{v,S'} := \bigcup_{I \subset S'} \left\{ \left(\prod_{g \in I} g \right) v \right\}$$

The i-faces of the complex are the collection of all the possible rooted spanned i-faces induced by $Cay(\mathcal{G}_n^d)$. We will denote them by $\mathcal{F}_i$.

Additionally, we name n and d, the side length and the dimension of the complex.

Definition 3.25 (Positive Edges and Cubes)**.** *For $u, v \in \mathcal{G}_n^d$, and $S_1', S_2' \subset [d]$, with sizes $|S_1'| = |S_2'| = i$, we say that the i-faces $\theta_{v,S_1'}$ and $\theta_{u,S_2'}$ are adjacent if the intersection $\theta_{v,S_1'} \cap \theta_{u,S_2'}$ is an $i-1$-face. In addition, if θ_1 and θ_2 are i and $i-1$-faces such that $\theta_2 \subset \theta_1$, then we say that θ_2 is a positive edge relative to face θ_1 if they are rooted at the same vertex. Furthermore, we say that θ_1 is a positive cube of θ_2. When the i-face is clear from the context, we will abbreviate and say that θ_2 is positive.*

Example 3.26. *For example, let $v, u \in \mathcal{G}_n^d$ and $g_1 \neq g_2 \in S$. Consider the 2-face $\theta = \{v, g_1 v, g_2 g_1 v, g_2 v\}$. Then, for $x, y \in \mathcal{G}_n^d$, we say that an edge (1-face) $\{x, y\}$ is positive relative to the face θ if $x = v$ and y is either $g_1 v$ or $g_2 v$.*

We denote the complex by $G = (V, E, F)$, where V, E, and F are the 0, 1, and 2 faces.

Although the present paper is not organized in the language of chain complexes, the boundary operators below are the standard ones for toric codes. We use them because they give a compact description of the local constraints and make the CSS structure transparent. We do not need the full homological formalism here, but it remains the natural background framework for the code family.

Definition 3.27 (Assignments on a toric complex)**.** *Let $\mathcal{T}$ be a d-dimensional toric complex. For each i, the set of i-faces induces a vector space $\mathbb{F}_2^{|\mathcal{F}_i|}$. An element $z \in \mathbb{F}_2^{|\mathcal{F}_i|}$ is called an assignment on the i-faces of $\mathcal{T}$.*

For a vertex v and a subset of generators $S' \subset S$ of size $|S'| = i$, let $\theta_{\mathbf{v},\mathbf{S}'}$ denote the standard basis vector corresponding to the i-face $\theta_{v,S'}$. Thus the family

$$\left\{ \theta_{\mathbf{v},\mathbf{S}'} : v \in \mathcal{F}_0,\ S' \subset S,\ |S'| = i \right\}$$

is the standard coordinate basis of $\mathbb{F}_2^{|\mathcal{F}_i|}$.

Let z be an assignment on the i-faces. Let $v_1, \ldots, v_k \in \mathcal{F}_0$ and $S'_1, \ldots, S'_k \subset S$ be such that $\theta_{v_1,S'_1}, \theta_{v_2,S'_2}, \ldots, \theta_{v_k,S'_k}$ are precisely the i-faces in the support of z, namely:

$$z = \sum_{i=1}^{k} \theta_{\mathbf{v_i},\mathbf{S'_i}}$$

We refer to $\theta_{v_1,S'_1}, \theta_{v_2,S'_2}, \ldots, \theta_{v_k,S'_k}$ as the collection of faces that form z, or briefly, the faces form z. For $i > 0$, we define the linear map $\partial : \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_{i-1}|}$ such that

$$\partial\theta_{\mathbf{v},\mathbf{S'}} := \sum_{g \in S'} \theta_{\mathbf{v},\mathbf{S'/g}} + \sum_{g \in S'} \theta_{\mathbf{gv},\mathbf{S'/g}}$$

For $i < d$, we define the linear map $\partial^* : \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_{i+1}|}$ such that

$$\partial^*\theta_{\mathbf{v},\mathbf{S'}} := \sum_{g \in S/S'} \theta_{\mathbf{v},\mathbf{S'\cup g}} + \sum_{g \in S'} \theta_{\mathbf{g^{-1}v},\mathbf{S'\cup g}}$$

We define the restriction of z to a vertex v to be the linear map $|_v \colon \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u},\mathbf{S'}}|_v := \begin{cases} \theta_{\mathbf{u},\mathbf{S'}} & v \in \theta_{u,S'} \\ 0 & else \end{cases}$$

We define the front of z to a vertex v to be the linear map $|_{v+} \colon \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_i|}$ such that:

$$\theta_{\mathbf{u},\mathbf{S'}}|_{v+} := \begin{cases} \theta_{\mathbf{u},\mathbf{S'}} & u = v \\ 0 & else \end{cases}$$

We say that an assignment z is connected if the graph that is induced by taking the faces that support z as vertices and defining the edges to be the adjacency relation between them is connected. We define the connected components of z to be its subsets such that each of them is a connected assignment. For a j-face f, we say that $f \in z$ if z has a support on $\theta_{\mathbf{v},\mathbf{S'}}$ such that $f \in \theta_{v,S'}$. Consider an assignment $z \in \mathbb{F}_2^{\mathcal{F}_i}$ and a vertex $v \in \mathcal{G}_n^d$. We say that z is a rooted assignment at vertex v if there are subsets $S'_1, \ldots, S'_k \subset S^{\times k}$ such that each of S'_j is at size i and z is decomposed as:

$$z = \sum_{j}^{k} \theta_{\mathbf{v},\mathbf{S'_j}}$$

Put differently, z is the result of taking a collection of i-faces that are rooted at v.

Claim 3.28 (CSS structure)**.** *For every i, the maps $\partial \colon \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_{i-1}|}$ and $\partial^* \colon \mathbb{F}_2^{|\mathcal{F}_i|} \to \mathbb{F}_2^{|\mathcal{F}_{i+1}|}$ satisfy the usual orthogonality relation between boundary and coboundary operators. Consequently, the X-checks defined by ∂ and the Z-checks defined by ∂^* commute, and therefore the resulting toric code is a CSS code.*

Proof. It is enough to check the claim on basis vectors. For a basis vector $\theta_{\mathbf{v},\mathbf{S}'}$, the appendix calculation shows that

$$\partial\big(\partial\theta_{\mathbf{v},\mathbf{S}'}\big) = 0.$$

The same cancellation argument applies dually to the coboundary operator. Equivalently, every $d'-1$ face and every $d'+1$ face overlap on an even number of d'-faces, so the corresponding X- and Z-checks commute. This is exactly the CSS condition. □

We will use only this CSS property and the local geometry of the code; standard facts about distance and rate may be found in the toric-code literature, for example in [Den+02].

Definition 3.29 ((d,d') Toric Code)**.** *Let d, d' be integers such that $1 < d' < d$. For any integer n, we construct the nth instance of the quantum code family (d,d') Toric Code as follows: Let $G = (\mathcal{F}_0, \ldots, \mathcal{F}_d)$ be the d-dimensional toric complex obtained from $\mathcal{G}_n^d$ as defined in Definition 3.24. We associate the (q)bits with the $\mathcal{F}_{d'}$, the X-checks with the $\mathcal{F}_{d'-1}$, and the Z-checks with the $\mathcal{F}_{d'+1}$. An X-check, c_x, is satisfied if the parity of (q)bits set on the d'-face containing c_x is even. Similarly, a Z-check, c_z, is satisfied if the parity, in the Hadamard basis, of the (q)bits set on the d'-faces contained in c_z is even.*

Claim 3.30. *For any generator subset S' of size d', denote by $x_{S'}$ and $z_{S'}$ the following assignments:*

$$x_{S'} := \sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}'} \qquad z_{S'} := \sum_{u\in\langle S/S'\rangle} \theta_{\mathbf{u},\mathbf{S}'}$$

Then $x_{S'}$ and $z_{S'}$ are binary representations of anticommuting logical codewords of the (d,d')-Toric code. In particular, $x_{S'} \in \mathcal{C}_X/\mathcal{C}_Z^{\perp}$, $z_{S'} \in \mathcal{C}_Z/\mathcal{C}_X^{\perp}$, and $x_{S'}z_{S'} = 1$. In addition, let $X_{S'}$ and $Z_{S'}$ be the Pauli matrices obtained by multiplying each non-trivial coordinate of them by X or Z, respectively.

Proof. By definition $x_{S'}$ and $z_{S'}$ are binary assignments on the d'-dimensional faces. We start by showing that $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$:

$$\begin{aligned}
\partial^- \sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}'} &= \sum_{u\in\langle S'\rangle}\sum_{g\in S'} \theta_{\mathbf{u},\mathbf{S}'/\mathbf{g}} + \theta_{\mathbf{gu},\mathbf{S}'/\mathbf{g}} \\
&= \sum_{g\in S'}\sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}'/\mathbf{g}} + \sum_{u\in\langle S'\rangle} \theta_{\mathbf{gu},\mathbf{S}'/\mathbf{g}} \\
&= \sum_{g\in S'}\sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}'/\mathbf{g}} + \sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}'/\mathbf{g}} = 0 \\
\partial^+ \sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S}'} &= \sum_{u\in\langle S'\rangle}\sum_{g\in S'} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} + \theta_{\mathbf{g}^{-1}\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} \\
&= \sum_{g\in S'}\sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} + \sum_{u\in\langle S'\rangle} \theta_{\mathbf{g}^{-1}\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} \\
&= \sum_{g\in S'}\sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} + \sum_{u\in\langle S'\rangle} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S}'\cup\mathbf{g}} = 0
\end{aligned}$$

Thus $x_{S'} \in \mathcal{C}_X$ and $z_{S'} \in \mathcal{C}_Z$, now for showing that they are not binary representations of stabilizers, it's enough to show they admit no vanishing product.

$$x_{S'} z_{S'} = \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u},\mathbf{S}'} \sum_{u \in \langle S/S' \rangle} \theta_{\mathbf{u},\mathbf{S}'} = \theta_{\mathbf{e},\mathbf{S}'} \theta_{\mathbf{e},\mathbf{S}'} = 1$$

□

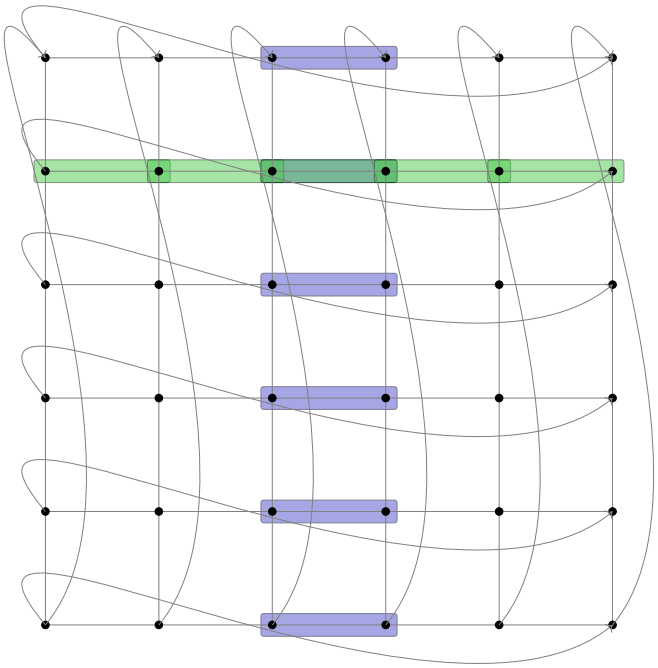

Figure 6: In blue and green are the binary presentations x_S and z_S of logical code words of the $(2,2)$-Toric code. The edge $\{(5,4),(0,4)\}$ is omitted for clarity (it should be a green edge that hovers over the green line).

Definition 3.31 (The Transpose Map)**.** *Let d, d' be integers, and let $\mathcal{G}$ be a (d,d')-Toric complex generated by the generator set S. We define the transpose map $\phi : \mathcal{F}^{d'} \to \mathcal{F}^{d-d'}$ by acting on the basis elements as follows:*

$$\phi(\theta_{\mathbf{v},\mathbf{S}'}) \to \theta_{\mathbf{v}^{-1},\mathbf{S}/\mathbf{S}'}$$

Notice that for $d' = d - d'$, the transpose map is a permutation over the bits.

Definition 3.32 (Hadamard-Type Gate H_ϕ)**.** *For $d' = d - d'$, we define the logical Hadamard-type gate H_ϕ: $L \to L$ as follows, for any $S' \subset S$, of size d':*

$$H_\phi(X_{S'}) \to Z_{S/S'}$$
$$H_\phi(Z_{S'}) \to X_{S/S'}$$

[**COMMENT**] Change the definition above such that it will be about moving to the dual code $C_X/C_Z^\perp \to C_Z/C_X^\perp$. And not about the logical Hadamard.

Claim 3.33. *The Hadamard-type gate H_ϕ has a fold-transversal implementation by, first applying H^n over all the physical qubits, and then permuting them by the transpose map ϕ, namely $\hat{H}_\phi = \phi \circ H^n$.*

Proof. Clearly, for any subset of generators $S' \subset S$, the map ϕ sends $x_{S'}$ to $z_{S/S'}$:

$$\phi(x_{S'}) = \phi(\sum_{u \in \langle S' \rangle} \theta_{\mathbf{u},\mathbf{S'}}) = \sum_{u \in \langle S' \rangle} \theta_{\mathbf{u},\mathbf{S}/\mathbf{S'}} = z_{S/S'}$$

Moreover, ϕ sends binary representations of Z-stabilizers to binary representations of X-stabilizers, and vice versa. To see this, consider a $d'+1$ dimensional face, and consider the image of the stabilizer it defines.

$$\begin{aligned}
\phi\left(\partial^- \theta_{\mathbf{v},\mathbf{S'}}\right) &= \phi\left(\sum_{g \in S'} \theta_{\mathbf{v},\mathbf{S'}/\mathbf{g}} + \theta_{\mathbf{gv},\mathbf{S'}/\mathbf{g}}\right) \\
&= \sum_{g \in S'} \theta_{\mathbf{v^{-1}},(\mathbf{S}/\mathbf{S'})\cup\mathbf{g}} + \theta_{\mathbf{g^{-1}v^{-1}},(\mathbf{S}/\mathbf{S'})\cup\mathbf{g}} \\
&= \partial^+ \theta_{\mathbf{v^{-1}},\mathbf{S}/\mathbf{S'}}
\end{aligned}$$

And since $\phi \circ \phi = I$, we also have that $\phi\left(\partial^+ \theta_{\mathbf{v^{-1}},\mathbf{S}/\mathbf{S'}}\right)$. Thus, $H^{\otimes n} \circ \phi$ sends Z-stabilizers to X-stabilizers and vice versa, thereby preserving the code space. Combining this with the above, $H^{\otimes n} \circ \phi$ realizes the logical form H_ϕ in the $(d, d/2)$-Toric code. □

Example 3.34. *Consider the $(2,1)$-Toric code. Name the generators in S by $g_1 = (1,0)$ and $g_2 = (0,1)$. Consider the check corresponding to the Z-stabilizer $\partial^+ \theta_{(\mathbf{2},\mathbf{3}),\{\}}$.*

$$\partial^+ \theta_{(\mathbf{2},\mathbf{3}),\{\}} = \{\theta_{(\mathbf{2},\mathbf{3}),\{(\mathbf{1},\mathbf{0})\}}, \theta_{(\mathbf{2},\mathbf{3}),\{(\mathbf{0},\mathbf{1})\}}, \theta_{(\mathbf{1},\mathbf{3}),\{(\mathbf{1},\mathbf{0})\}}, \theta_{(\mathbf{2},\mathbf{2}),\{(\mathbf{0},\mathbf{1})\}}\}$$

The ϕ-transpose map acts on it as follows:

$$\begin{aligned}
\phi(\theta_{(\mathbf{2},\mathbf{3}),\{(\mathbf{1},\mathbf{0})\}}) &= \theta_{(\mathbf{4},\mathbf{3}),\{(\mathbf{0},\mathbf{1})\}} \\
\phi(\theta_{(\mathbf{2},\mathbf{3}),\{(\mathbf{0},\mathbf{1})\}}) &= \theta_{(\mathbf{4},\mathbf{3}),\{(\mathbf{1},\mathbf{0})\}} \\
\phi(\theta_{(\mathbf{1},\mathbf{3}),\{(\mathbf{1},\mathbf{0})\}}) &= \theta_{(\mathbf{5},\mathbf{3}),\{(\mathbf{0},\mathbf{1})\}} \\
\phi(\theta_{(\mathbf{2},\mathbf{2}),\{(\mathbf{0},\mathbf{1})\}}) &= \theta_{(\mathbf{4},\mathbf{4}),\{(\mathbf{1},\mathbf{0})\}}
\end{aligned}$$

What gives edges on a rectangle that converges with the bits on the support of the X-stabilizer $\theta_{(\mathbf{4},\mathbf{3}),\{(\mathbf{0},\mathbf{1}),(\mathbf{1},\mathbf{0})\}}$.

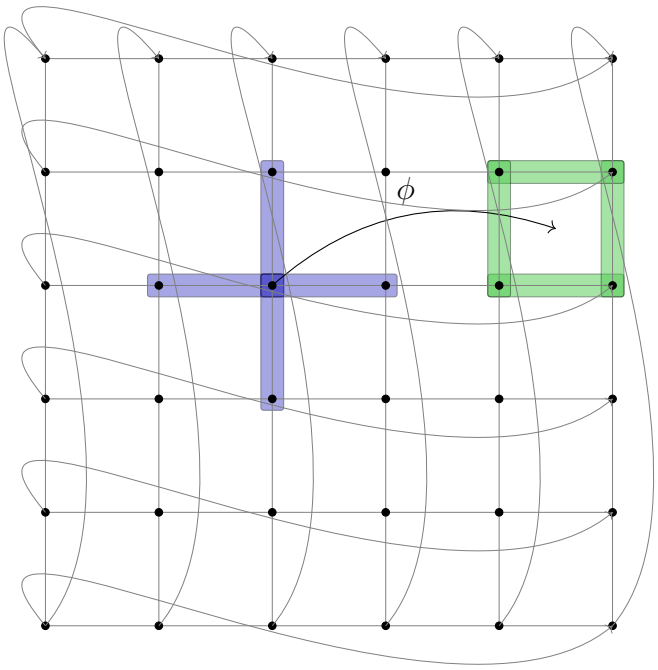

Figure 7: An outline of Example 3.34. The ϕ-transpose sends Z-stabilizers to X-stabilizers and vice versa. Here we consider the $(2,1)$ Toric code; namely, the Z-stabilizers (parity checks) are set on the vertices, and the X-stabilizers (phase checks) are set on the squares. In blue and green, we have the edges corresponding to checks $\partial^{+}\theta_{(\mathbf{2},\mathbf{3}),\{\}}$ and $\theta_{(\mathbf{4},\mathbf{3}),\{(\mathbf{0},\mathbf{1}),(\mathbf{1},\mathbf{0})\}}$. The example shows that indeed ϕ maps between the two.

[COMMENT] Here, add that the measurement in the X-basis can be done using $\mathbf{H}^n$, measurement, and then $\log(n)$ classical depth.

[COMMENT] Add a corollary that state that we can perform a gate teleportations of $\mathbf{S}$ and $\mathbf{H}$, with the resource states $|\mathbf{S}\rangle$ and $|\mathbf{H}\rangle$, $\log(n)$ classical depth, and only transversal operation over the encoded register.

Definition 3.35 (The Nice Encoding)**.** *Consider $(d, d/2)$-Toric code and fix a subset $S' \subset S$, at size d'. We define the nice encoding to be the subsystem code, which uses $x_{S'}$ to encode a single logical qubit. Namely, the logical codewords are:*

$$|\bar{0}\rangle = \left|\mathcal{C}_Z^{\perp}\right\rangle$$
$$|\bar{1}\rangle = \left|x_{S'} + \mathcal{C}_Z^{\perp}\right\rangle$$

Definition 3.36 (Decoding Gadget)**.** *Let C be a quantum circuit, and let u be a gate*

in C with fan-in at most Δ. We say that u is a decoding gadget if it has the following form:

1. *The wires of u wires can be decomposed into two subsets A and B such that the wires in A are reset wires, and there exists an encoded register R in C such that the wires in B are all set in R.*

2. *For a set of faults $\mathcal{E}$, there is a Pauli gate u' that acts only on B such that the action of u in $C[\mathcal{E}]$ equals u'. Put differently, the quantum circuit obtained from $C[\mathcal{E}]$ by replacing u with u' equals $C[\mathcal{E}]$.*

Definition 3.37 (Unitary Decoding Representation)**.** *Let $C = \{u_i\}_{i=1}$ be a quantum circuit, and let $\mathcal{U} = \{u'_i\}_{i=1} \subset C$ be a subset of the gates in C that are decoding gadgets. For a set of faults $\mathcal{E}$, we define the unitary decoding representation of $C[\mathcal{E}]$ to be the quantum circuit obtained by replacing each of the gates in $\mathcal{U}$ with their corresponding Pauli gate according to $\mathcal{E}$.*

Definition 3.38 (Register coordinate system)**.** *Let $C = \{u_i\}_i$ be a quantum circuit of width w with registers $\mathcal{R} = R_1, \ldots, R_k$, each of length at most n. For each register R_i, define its identifier by $\mathbf{1}_{R_i} = \mathbf{1}_i$. We define the map*

$$\phi_{\mathcal{R}} \colon \mathbb{Z}_w \to \mathbb{Z}_R \times \mathbb{Z}_n$$

by setting

$$\phi_{\mathcal{R}}(x) = (\mathbf{1}_{R_j}, x'),$$

where R_j is the register containing x, and x' is the position of that qubit within R_j.

The presentation of C in the register coordinate system is obtained by replacing every spatial coordinate by its image under $\phi_{\mathcal{R}}$. Thus, if $u_i = (g_i, l_i) \in C$ and the space-location part of l_i is $l_{i,2} = (l_{i,2}^{(1)}, \ldots, l_{i,2}^{(\Delta)})$, then we replace it by

$$(\phi_{\mathcal{R}}(l_{i,2}^{(1)}), \ldots, \phi_{\mathcal{R}}(l_{i,2}^{(\Delta)})).$$

We call the first coordinate of $\phi_{\mathcal{R}}(x)$ the register coordinate and the second the inner coordinate.

3.4 Infinite Torics

Definition 3.39 (Shifting in Registers System)**.** *Consider the register coordinate system induced by a quantum circuit of width w, with R registers, each of length at most n. For an integer $r \in \mathbb{Z}$, we define the r-shifting function $Sh_r \colon (\mathbb{Z}_R \times \mathbb{Z}_n)^{\times\Delta} \to (\mathbb{Z}_R \times \mathbb{Z})^{\times\Delta}$ to act on space-location coordinates as follows:*

$$Sh_r(l_1, .., l_k) = (Sh_r(l_1), .., Sh_r(l_k))$$

$$Where\ Sh_r(l_i) = \begin{cases} l_i + r & \text{if } l_i \text{ is an inner coordinate} \\ l_i & \text{else} \end{cases}$$

Definition 3.40. *Let n be an integer, and let $C = \{u_i\}_{i=1}$ be a quantum circuit with registers $\mathcal{R} = R_1, \ldots, R_k$ such that each register R_i is a toric encoding with a side length n. In addition, all the decoding gadgets of C are sweep rule gadgets; let us denote them by $\mathcal{U}$. We define the ∞-extension of C, denoted by C^∞, as follows:*

1. *For a register $R_i \in \mathcal{R}$, denote by d_i, d_i' the dimension of the toric complex that induces the code, and the dimension of the faces that are associated with the bits. Each register R_i in $\mathcal{R}$ is mapped to a register R_i', which is a (d_i, d_i')-dimensional, infinite side length, toric encoding. Since the mapping between registers is one-to-one, we use the identifier $\mathbf{1}_{R_i}$ to identify also R_i'.*

2. *Each gate $u_i \in C/\mathcal{U}$ is mapped to:*

$$u_i = (g_i, l_i) \mapsto \bigcup_{j=-\infty}^{\infty} \{(g_i, (Sh_{jn}(l_i))\}$$

3. *Each gate $u \in \mathcal{U}$ is mapped to the collection of sweep rule gadgets $\{v_j\}_{j=-\infty}^{\infty}$ such that each of them has the same time coordinate as u, is in the register that corresponds to the image of u's register, and is rooted at $root(v) + jn$.*

For a set of faults $\mathcal{E} = \{f_i\}_{i=1}$, we denote by $C^\infty[\mathcal{E}^\infty]$ the ∞-extension of $C[\mathcal{E}]$.

Claim 3.41. *Let $u = (g, l)$, and let $v_0, v_{\pm 1}$ be its translates in the ∞-extension rooted at $\mathrm{root}(u)$ and $\mathrm{root}(u) \pm n$. Then*

$$v_0 v_{\pm 1}|_{[w]} = u.$$

Proof. The gates v_0 and $v_{\pm 1}$ are copies of the same local Pauli operator acting on translates of the same finite pattern. When restricted to the fundamental domain $[w]$, all factors outside $[w]$ disappear, and the remaining action coincides with the original gate u. Thus the product of the relevant translates restricts exactly to u. □

Definition 3.42. *For a quantum circuit C, of the form defined in Definition 3.40, and a set of faults $\mathcal{E}$, denote:*

$$\rho := C[\mathcal{E}] \left|0\right\rangle\!\left\langle 0\right| C[\mathcal{E}]^\dagger$$
$$\rho_\infty := C^\infty[\mathcal{E}^\infty] \left|0\right\rangle\!\left\langle 0\right| (C^\infty[\mathcal{E}^\infty])^\dagger$$

We say that a quantum circuit C is ∞-compatible if, for any set of faults $\mathcal{E} = \{f_i\}_{i=1}$, it holds that:

$$\rho = \mathbf{Tr}_{/[w]}(\rho_\infty)$$

Claim 3.43. *Let C be quantum circuit of the form defined in Definition 3.40. Then C is ∞-compatible.*

Proof. It is enough to prove the claim for circuits in their unitary decoding representation. We prove it by induction on the number of the gates.

1. Base. For an empty circuit, ρ_∞ is just the infinite string of zeros; thus, the restriction of it to $[w]$ locations, namely $\mathbf{Tr}_{/[w]}(\rho_\infty)$, yields a single matrix element corresponds to the w-length string of zeros, which is exactly ρ.

2. Step. Assume the claim holds for every circuit with fewer than τ gates, and consider a circuit C with τ gates. Denote by C' the circuit obtained from C by erasing its last gate; then C' has $\tau - 1$ gates. Denote by ρ' and ρ'_∞ the output states of $C'[\mathcal{E}]$ and $(C')^\infty[\mathcal{E}^\infty]$, respectively. By the induction assumption we have:

$$\rho' = \mathbf{Tr}_{/[w]}\left(\rho'_\infty\right)$$

Denote the last gate in C by u, and by $I = \{v_j\}_{-\infty}^{\infty}$ the collection of gates in C^∞ to which u is mapped. Let us split into cases, depending on whether u is or is not a decoding gate.

(a) Suppose that u is not a decoding gate. Let g and l be the gate element and the location of u. By the definition of ∞-extension, I equals:

$$I = \bigcup_{j=-\infty}^{\infty} \{(g, (\mathrm{Sh}_{jn}(l))\}$$

So there is only one gate in I, corresponding to $j = 0$, that has support on qubits with spatial location in the range $[w]$. Denote that gate by v_0.

$$\begin{aligned}
\mathbf{Tr}_{/[w]}\left(\rho_\infty\right) &= \mathbf{Tr}_{/[w]}\left(\prod_{j=-\infty}^{\infty} v_j \rho'_\infty \prod_{j=-\infty}^{\infty} v_j^\dagger\right) \\
&= v_0 \mathbf{Tr}_{/[w]}\left(\prod_{j=-\infty, j\neq 0}^{\infty} v_j \rho'_\infty \prod_{j=-\infty, j\neq 0}^{\infty} v_j^\dagger\right) v_0^\dagger \\
&= v_0 \mathbf{Tr}_{/[w]}\left(\prod_{j=-\infty, j\neq 0}^{\infty} v_j^\dagger \prod_{j=-\infty, j\neq 0}^{\infty} v_j \rho'_\infty\right) v_0^\dagger \\
&= v_0 \mathbf{Tr}_{/[w]}\left(\rho'_\infty\right) v_0^\dagger \\
&= v_0 \rho' v_0^\dagger \\
&= \rho
\end{aligned}$$

(b) In the case that u is a decoding gate, observe that there are at most two gates in I that have support on the qubits with space-location in $[w]$. If there is exactly one, then repeating the non-decoding gate case while setting this gate as v_0 gives the desired result. Else, in the case where two gates act non-trivially on qubits at $[w]$, denote those gates by v_0, v_1. Since we assumed that C is given in its unitary decoding representation, it holds that v_0, v_1 are Paulis, in particular a product operator. Denote their product

decompositions by $v_0 = \prod_k v_0^{(k)}$ and $v_1 = \prod_k v_1^{(k)}$. Thus, by repeating the non-decoding case, but extracting from the infinite product only the terms in v_0 and v_1 that act non-trivially at $[w]$, we get:

$$\mathbf{Tr}_{/[w]}(\rho_\infty) = v_0 v_1|_{[w]} \rho' \left(v_0 v_1|_{[w]}\right)^\dagger$$

Moreover, since v_0 and v_1 are translates of u in locations $\mathrm{root}(u)$ and $\mathrm{root}(u) \pm n$, then by Claim 3.41, we have that $u = v_0 v_1|_{[w]}$; therefore:

$$\begin{aligned}\mathbf{Tr}_{/[w]}(\rho_\infty) &= v_0 v_1|_{[w]} \rho' \left(v_0 v_1|_{[w]}\right)^\dagger \\ &= u \rho' u^\dagger \\ &= \rho\end{aligned}$$

□

The computation DAG lifts naturally to the ∞-extension: gates that arise as translates of the same local gate act on disjoint translated supports inside different periods, so they may be executed simultaneously. The same periodic lifting applies to the underlying geometric objects, passing from the finite toric complex to its infinite cover.

Definition 3.44 (The standart embedding)**.** *Consider the d-dimensional infinite grid $\mathcal{G}^d$. The standard embedding $\mathcal{G}^d \to \mathbb{R}^d$ is given by fixing a vertex in $\mathcal{G}^d$ to be sent to the origin and then mapping the generators of the grid to the standard basis. We extend notation as follows: for a vertex $v \in \mathcal{G}^d$, a generator $g \in S$, and a coordinate $i \in [d]$, such that under the embedding $g \to i$, we denote by $v_i = v_g$ the i'th coordinate of the image of v.*

Definition 3.45 (Potential Walls)**.** *We define the potential walls to be the $d+1$ functions $L_1, L_2, .., L_{d+1}$, from the vertices of $\mathcal{G}_n^d$ to the reals as follow:*

1. *For any coordinate $i \in [d]$, we define $L_i \colon \mathcal{G}^d \to \mathbb{R}$ as $L_i(v) := v_i$. In addition, for the generator $g \in S$ which is mapped to i, i.e $g \to i$, we identify $L_g = L_i$.*
2. *We define $L_{d+1} \colon \mathcal{G}^d \to \mathbb{R}$ by $L_{d+1}(v) := \sum_{i=1}^d v_i$.*

Consider an assignment z over the faces. For a vertex $v \in z$, and a potential wall $l \in \{L_1, .., L_{d+1}\}$, we say that v is a local maximum of l in z if for any neighbor u of v, such that $u \in z$, we have $l(v) \geq l(u)$. We say that v is a strong local maximum if the inequality is strict, namely $l(v) > l(u)$. When the assignment z is clear from the context, we might omit it and briefly say that v is a local maximum of l.

With the assistance of potential walls, we can discuss the endpoint of a bunch of bits, which will soon become error clusters.

3.5 Sweep Rule

Definition 3.46 (Sweep Rule)**.** *Let z be an assignment on the d'-faces. The sweep rule decoding procedure at a vertex $v \in \mathcal{G}_n^d$ is defined as follows:*

Measure the checks rooted at v and look for a rooted assignment $\tilde{z}$ at v such that $(\partial\tilde{z})|_{v+} = \partial(z)|_{v+}$. In other words, the X-checks rooted at v have the same syndromes for z and $\tilde{z}$. If such an assignment exists, flip $\tilde{z}$.

Definition 3.47 (Sweep cycle)**.** *We define the sweep cycle to be the following procedure:*

1. *For each vertex, perform the sweep rule.*
2. *Apply the Hadamard transform $\mathbf{H}^{\otimes n}$ followed by the ϕ-transpose, and flip the positive direction.*
3. *Again, apply the sweep rule at each vertex.*
4. *Reverse stage 2 by reordering the qubits according to the ϕ-transpose, and then apply the Hadamard transform $\mathbf{H}^{\otimes n}$. Flip the positive direction again.*

Claim 3.48. *The local gates that perform the sweep rule are decoding gadgets.*

Proof. A local sweep update acts on a constant-size neighborhood around a vertex, uses only local syndrome information, and may be implemented with fresh ancillas that are reset after the update. Any dependence on faults inside the gadget can therefore be pushed onto the data wires as an effective Pauli operator supported on the encoded register, while the reset ancillas carry no residual information. This is exactly the form required in the definition of a decoding gadget. □

3.6 Shrinking

The following two claims relate the local maximum points of $L_1, \ldots, L_d$ in an assignment z to the structure of the syndrome they observe. Later, we will use that relation to infer that the decoding step of the SWEEP rule can't spread the error to a vertex[6] with a higher value than the current global maximum of $L_1, \ldots, L_d$. In other words, the error does not spread in the positive directions.

Claim 3.49. *Let z be an assignment, and recall that the vertices in ∂z are the set of vertices adjacent to unsatisfied checks induced by z. Let v be a vertex in ∂z, and let g be a generator in S such that v is a local maximum of L_g in ∂z. Then $gv \notin \partial z$.*

Proof. Suppose $gv \in \partial z$, then

$$L_g(gv) = (gv)_g = 1 + v_g = 1 + L_g(v)$$

In contradiction to v being a local maximum of L_g in ∂z. □

[6] To faces whose vertices have higher values of $L_1, \ldots, L_d$ than the current global maximum values.

Claim 3.50. *Let z be an assignment, let v be a vertex in ∂z, and let g be a generator in S such that v is a local maximum of L_g. Then, for any check associated with a $d'-1$ face rooted at v that contains gv, it is satisfied.*

Proof. Assume, through contradiction, the existence of an unsatisfied check associated with a $d'-1$ positive face rooted at v, which contains gv. This implies $gv \in \partial z$, contradicting Claim 3.49. □

For the $(d+1)$th potential wall, we would like to have a stronger claim that implies the error cluster shrinks. For this, we start by showing that any error is equivalent, up to stabilizer addition, to an error whose left-corner edge sees a syndrome. We present two notations of local codes that describe the local view of a $(d+1)$th edge that sees no syndrome. They are the *stabilizers code at vertex v* and the *small code at vertex v*. The first corresponds to stabilizers rooted at the vertex, while the second is the collection of local views on its positive faces that satisfy its positive checks. Then, in Claim 3.53, we consider a vertex which is a $(d+1)$th edge and doesn't see a syndrome, meaning that its local view is in its *small code*, and show that there is a codeword of its *stabilizers code* that agrees with its positive faces. Addition by that stabilizer yields a new assignment excluding v.

Definition 3.51 (Stabilizers Code at Vertex v)**.** *We define stabilizer code at vertex v, denoted by $Stab_v$, as the restriction of the (d,d')-Toric code to the d'-dimensional faces such that their vertices are at a distance of at most one positive step in each direction from v. That is equivalent to the union of all the positive d'-dimensional faces of v with the d-dimensional faces of each of its sibling gv, excluding faces containing g as their generator. Namely:*

$$\begin{aligned} Stab_v := \big\{ z : \partial^- z = 0 \ \textit{and} \\ z \in Span\big(\big\{ \theta_{\mathbf{v},\mathbf{S'}} : S' \subset S, |S'| = d' \big\} \\ \bigcup_{g \in S} \big\{ \theta_{\mathbf{gv},\mathbf{S'/g}} : S' \subset S/g, |S'| = d' \big\} \big) \big\} \end{aligned}$$

The checks of the code, are of the following three types:

1. Checks that are rooted at v, each identified by choosing $d-1$ generators from S. Namely:

   $$\big\{ \theta_{\mathbf{v},\mathbf{S'}} : S' \subset S, |S'| = d'-1 \big\}$$

   For each check, the bits that connect to it, are given by restricting the coboundary of the check to bits domain of Stab_v:

   $$\big(\partial \theta_{\mathbf{v},\mathbf{S'}}\big)|_{\text{Stab}_v} = \big(\sum_{g \in S/S'} \theta_{\mathbf{v},\mathbf{S'} \cup \mathbf{g}} + \sum_{g \in S'} \theta_{\mathbf{g^{-1}v},\mathbf{S'} \cup \mathbf{g}} \big)|_{\text{Stab}_v} = \sum_{g \in S/S'} \theta_{\mathbf{v},\mathbf{S'} \cup \mathbf{g}}$$

2. Checks are rooted at the positive sibling of v. Consider such a sibling, let's say gv (namely the vertex that is given by stepping in the direction $g \in S$ from v).

Notice that any face rooted at gv that contains ggv cannot be contained in faces rooted at v. Thus, the checks of gv correspond to choosing subsets from S/g. Namely:

$$\bigcup_{g\in S}\left\{\theta_{\mathbf{gv},\mathbf{S'}} : S'\subset S/g, |S'| = d'-1\right\}$$

And each check is supported by:

$$\begin{aligned}\left(\partial\theta_{\mathbf{gv},\mathbf{S'}}\right)|_{\mathrm{Stab}_v} &= \Big(\sum_{g'\in S/S'}\theta_{\mathbf{gv},\mathbf{S'}\cup\mathbf{g'}} + \sum_{g'\in S'}\theta_{\mathbf{g'^{-1}gv},\mathbf{S'}\cup\mathbf{g'}}\Big)|_{\mathrm{Stab}_v}\\ &= \Big(\sum_{\substack{g'\in S/S'\\ g'\neq g}}\theta_{\mathbf{gv},\mathbf{S'}\cup\mathbf{g'}}\Big) + \theta_{\mathbf{v},\mathbf{S'}\cup\mathbf{g}}\end{aligned}$$

3. Checks that their roots are at a two-step distance from v, namely vertices of the form $g, g' \in S$, where $g \neq g'$. By the arguments presented in the second type, the checks are the $d'-1$ faces which do not have the generators on the path leading from v to them, i.e., g, g', in their generator set.

$$\bigcup_{g,g' in S'}\left\{\theta_{\mathbf{g'gv},\mathbf{S'}} : S'\subset S/\{g,g'\}, |S'| = d'-1\right\}$$

And for each check, its support is:

$$\begin{aligned}\left(\partial\theta_{\mathbf{g'gv},\mathbf{S'}}\right)|_{\mathrm{Stab}_v} &= \Big(\sum_{g''\in S/S'}\theta_{\mathbf{gg'v},\mathbf{S'}\cup\mathbf{g''}} + \sum_{g\in S'}\theta_{\mathbf{g''^{-1}gg'v},\mathbf{S'}\cup\mathbf{g''}}\Big)|_{\mathrm{Stab}_v}\\ &= \theta_{\mathbf{gv},\mathbf{S'}\cup\mathbf{g'}} + \theta_{\mathbf{g'v},\mathbf{S'}\cup\mathbf{g}}\end{aligned}$$

Definition 3.52 (Small Code at Vertex v)**.** *For a vertex v, we denote by $\mathcal{C}_v$ the small code at vertex v, which is defined to be the binary assignments over the bits (d' faces) rooted at v that satisfy the positive checks of v, namely:*

$$\begin{aligned}\mathcal{C}_v := \big\{z : \partial^- z = 0 \text{ and }\\ z\in \mathit{Span}\big(\left\{\theta_{\mathbf{v},\mathbf{S'}} : S'\subset S, |S'| = d'\right\}\big)\big\}\end{aligned}$$

The checks of the code are exactly the first type checks of Stab_v.

Claim 3.53. *Assume $d = 4$ and $d' = 2$. Let v be a vertex. For any $z \in \mathcal{C}_v$, there is a stabilizer $w \in \mathit{Stab}_v$ such that $z = w|_v$; namely, z and w agree on the faces rooted at v.*

Proof. We argue that the top six rows of the generating matrix of Stab_v are exactly the generating matrix of $\mathcal{C}_v$. This means that any codeword of Stab_v is an extension of a codeword of $\mathcal{C}_v$. Denote by $H_{\mathcal{C}_v}$ and H_{Stab_v} the parity check matrices of $\mathcal{C}_v$ and

Stab_v respectively. Similarly, denote by $G_{\mathcal{C}_v}$ and G_{Stab_v} their generating matrices. The parity check matrices are:

$$H_{\mathcal{C}_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \qquad H_{\mathrm{Stab}_v} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Where we mark in blue, brown, and red the first, second, and third types of checks.

$$G_{\mathcal{C}_v} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \qquad G_{\mathrm{Stab}_v} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

□

Conjecture 3.54. *For any integers d and $d' < d$, $G_{\mathcal{C}_v}$ is contained in G_{Stab_v}. In particular, Claim 3.53 holds for any dimension.*

Claim 3.55. *Let z be a finite assignment. Suppose that*

$$\max\left\{L_{d+1}(u) : u \in z\right\} > \max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\}$$

Then there is $\tilde{z} \sim_{\mathcal{C}_X} z$ such that:

$$\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} \leq \max\left\{L_{d+1}(u) : u \in z\right\} - 1$$

Proof. Let $v_1, v_2, \ldots, v_k$ be the vertices in z that achieve the maximum for L_{d+1}, namely $v_1, v_2, \ldots, v_k = \arg\max_{v \in z} L_{d+1}(v)$, and assume that $v_i \notin \partial^{\pm} z$ for any $i \in [k]$.

Consider the ith vertex v_i. By definition, since $v_i \notin \partial^{\pm} z$, we have $\partial^{\pm} z|_{v_i} = 0$, and since v_i is a maximum point of L_{d+1} in z, it is also a local maximum, and therefore $z|_{v_i+} = z|_{v_i}$. Combining the two, we get that $z|_{v_i}$ is a codeword of the small code at v_i, namely $z|_{v_i} \in \mathcal{C}^{\pm}_{v_i}$. Thus, by Claim 3.53, there is a stabilizer $w_i \in \mathrm{Stab}^{\pm}_{v_i}$ such that w_i and z agree on the positive faces of v_i, namely $w_i|_{v_i} = z|_{v_i}$.

Set $\tilde{z} \leftarrow z + \sum_i w_i$. Since $\sum_i w_i$ is a stabilizer, we have $\tilde{z} \sim_{C_X} z$. In addition, for any $j \neq i$, it follows that $v_j \notin w_i$. Otherwise, there exists $S' \subset S$ such that $v_j \in \theta_{v_i, S'}$, and therefore $L_{d+1}(v_j) > L_{d+1}(v_i)$, in contradiction to v_i and v_j being both maximum points for L_{d+1} in $\partial^{\pm} z$. Thus, we have:

$$\tilde{z}|_{v_i} = (z + \sum_i w_i)|_{v_i} + (z + w_i)|_{v_i} = 0$$

Namely, for any i, the ith vertex v_i is not in $\tilde{z}$. Hence, the vertex domain of $\tilde{z}$ is contained in the union of the vertices of z with the vertices of $w_1, w_2, \ldots, w_k$, substituting $v_1, v_2, \ldots, v_k$. Overall, we get:

$$\begin{aligned}\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} &\leq \max\left\{L_{d+1}(u) : u \in z \bigcup_i w_i / \bigcup_i v_i\right\} \\ &\leq \max\left\{L_{d+1}(u) : u \in z\right\} - 1\end{aligned}$$

□

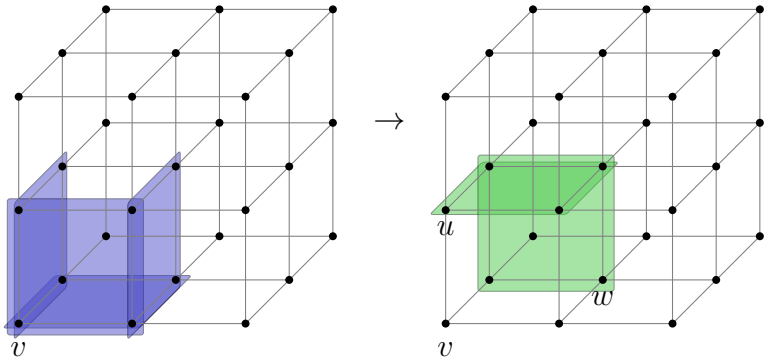

Figure 8: In blue, we have an assignment where the vertex achieving the minimum of L_{d+1}, denoted by v, does not see any unsatisfied checks in its positive local view. The green faces represent an assignment, equivalent to the blue up to stabilizer, such that any local maximum of L_{d+1} (vertices u and w) sees a non-trivial syndrome in its positive local view.

Claim 3.56. *Let z be a finite assignment. Then there is $\tilde{z} \sim_{\mathcal{C}_X} z$ such that*

$$\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} = \max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\}$$

Proof. Let z be a finite assignment, and let k be an integer such that $k \geq 1$ and

$$\max\left\{L_{d+1}(u) : u \in z\right\} = \max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\} + k$$

For an integer $l \leq k$, consider the assignments $\tilde{z}_0, \tilde{z}_1, .., \tilde{z}_l$ defined as follow:

- The first element, $\tilde{z}_0 = z$.
- If the $i-1$th element satisfies

  $$(\star) \quad \max\left\{L_{d+1}(u) : u \in \tilde{z}_{i-1}\right\} > \max\left\{L_{d+1}(u) : u \in \partial^{\pm} \tilde{z}_{i-1}\right\}$$

  Then define the ith element, $\tilde{z}_i$, to be the result of applying the construction, given in Claim 3.55, on $\tilde{z}_{i-1}$, namely $\partial \tilde{z}_{i+1} = \partial z_i$ and

  $$\max\left\{L_{d+1}(u) : u \in \tilde{z}_i\right\} \leq \max\left\{L_{d+1}(u) : u \in \tilde{z}_{i-1}\right\} - 1$$

  If the $i-1$th element doesn't satisfies $\star$, then set $l = i - 1$.

- If the kth element was defined, set $l = k$.

Since $\sim_{C_X}$ is a transitive relation, we have that for any $i \in [l]$, $z \sim_{C_X} \tilde{z}_i$. If $l \neq k$, then there exists $\tilde{z}_i$ such that $\tilde{z}_i \sim_{C_X}$ and $\tilde{z}_i$ satisfies $\star$, namely by setting $\tilde{z} = \tilde{z}_i$, we get the desired. So, it is left to handle the case where $l = k$. In that case, set $\tilde{z} = \tilde{z}_k$. By Claim 3.55:

$$\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} \leq \max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\}$$

Yet, since $\tilde{z} \sim_{C_X} z$, it follows that $\partial\tilde{z} = \partial z$, So:

$$\max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\} = \max\left\{L_{d+1}(u) : u \in \partial^{\pm}\tilde{z}\right\} \leq \max\left\{L_{d+1}(u) : u \in z\right\}$$

Therefore, we have the equality:

$$\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} = \max\left\{L_{d+1}(u) : u \in \partial^{\pm} z\right\}$$

□

Claim 3.57. *Let z be a finite assignment and let ξ be the result of the application of the Sweep rule on it. Then:*

1. *For any $i \in [d]$:* $\max\left\{L_i(v) : v \in \partial z\right\} = \max\left\{L_i(v) : v \in \partial\xi\right\}$
2. *And for $i = d+1$:* $\max\left\{L_{d+1}(v) : v \in \partial z\right\} > \max\left\{L_{d+1}(v) : v \in \partial\xi\right\} + 1$

Proof. First, since the Sweep rule is invariant to addition by stabilizers, for any $\tilde{z}$ such that $\tilde{z} \sim_{C_X} z$, the correction applied by the Sweep rule on $\tilde{z}$ is the same as the application on z. Denote by ϕ the correction, by $\tilde{\xi}$ the result of applying the Sweep rule on $\tilde{z}$, and by w the stabilizer which is the difference between z and $\tilde{z}$, namely:

$$\begin{aligned}\xi &= z + \phi \\ \tilde{\xi} &= \tilde{z} + \phi \\ \tilde{z} &= z + w\end{aligned}$$

The $\partial^{\pm}$ boundary of $\tilde{\xi}$ equals:

$$\partial\tilde{\xi} = \partial\left(\tilde{z} + \phi\right) = \partial\left(z + \phi\right) = \partial\xi$$

So, in total, we have that for any $\tilde{z}$ such that $\tilde{z} \sim_{C_X} z$, it holds that $\partial\tilde{z} = \partial z$ and $\partial\tilde{\xi} = \partial\xi$. Therefore, showing that there exists $\tilde{z} \sim_{C_X} z$ for which the claim holds proves the claim for z. By Claim 3.56, there is $\tilde{z} \sim_{C_X} z$ such that:

$$\max\left\{L_{d+1}(u) : u \in \tilde{z}\right\} = \max\left\{L_{d+1}(u) : u \in \partial^{\pm}\tilde{z}\right\}$$

For such $\tilde{z}$, any maximum of L_{d+1} in $\partial\tilde{z}$, is also a maximum in $\tilde{z}$. Denote by $v_1, .., v_k$ the maximum points in $\partial\tilde{z}$. Since they are maximum points in $\tilde{z}$, it holds that for each $i \in [k]$, $\tilde{z}|_{v_i} = \tilde{z}|_{v_i+}$, and hence $\partial\tilde{z}|_{v_i} = \partial\tilde{z}|_{v_i+}$. Since $\tilde{z}|_{v_i+}$ is an assignment

rooted at v_i, then v_i can suggest $\tilde{z}|_{v_i+}$, namely the condtion in Sweep rule is satisfied for vertex v_i. Thus:

$$\partial\tilde{\xi}|_{v_i} = \partial\tilde{z}|_{v_i} + \partial\phi|_{v_i} = 0$$

Namely, for any i, the ith vertex v_i is not in $\tilde{\xi}$. For any vertex v denote by ϕ_v the correction made by the vertex v, namely $\phi = \sum_{v\in\partial\tilde{z}} \phi_v$. Hence, the vertex domain of $\tilde{\xi}$ is contained in the union of the vertices of $\tilde{z}$ with the vertices of $\cup_{v\in V}\phi_v$, substituting maximum points $v_1, v_2, \ldots, v_k$. Overall, we get:

$$\begin{aligned}\max\left\{L_{d+1}(u) : u \in \partial\tilde{\xi}\right\} &\leq \max\left\{L_{d+1}(u) : u \in \partial\tilde{z} \bigcup_{v\in\partial\tilde{z}} \phi_v / \bigcup_i v_i\right\} \\ &\leq \max\left\{L_{d+1}(u) : u \in \partial\tilde{z}\right\} - 1\end{aligned}$$

Let v be a vertex that is a maximum of L_g in $\partial\tilde{z}$, maximum points of L_g do not see syndrome at the g direction. If and:

$$\phi_v = \sum \theta_{\mathbf{v},\mathbf{S_i'}}$$

By Claim 3.50 $\partial\tilde{z}|_{gv} = 0$, Hence:

$$\begin{aligned}\partial\phi_v|_{gv} &= \left(\partial \sum \theta_{\mathbf{v},\mathbf{S_i'}}\right)|_{gv} \\ &= \left(\sum_{S_i'}\sum_{g'\in S_i'} \theta_{\mathbf{v},\mathbf{S_i'/g'}} + \theta_{\mathbf{g'v},\mathbf{S_i'/g'}}\right)|_{gv} \\ &= 0\end{aligned}$$

Thus if $g \in \cup S_i'$, Then:

$$\theta_{\mathbf{gv},\mathbf{S_i'/g}} \in \partial\phi_v|_{gv}$$

So for $\theta_{\mathbf{gv},\mathbf{S_i'/g}} = 0$, it must appear at least twice in the sum, yet the left terms in the sum cannot contain it (they differ by the root vertex), and if there is a right term $\theta_{\mathbf{gv},\mathbf{S_j'/g}} = \theta_{\mathbf{gv},\mathbf{S_i'/g}}$, then it implies that $S_j = S_i$. Which is a contradiction, therefore: $g \notin \cup S_i'$ and $gv \notin \phi_v$.

$$\begin{aligned}\max\left\{L_g(u) : u \in \partial\tilde{\xi}\right\} &\leq \max\left\{L_g(u) : u \in \partial\tilde{z} \bigcup_{v\in\partial\tilde{z}} \phi_v / \bigcup_i v_i\right\} \\ &\leq \max\left\{L_g(u) : u \in \partial\tilde{z}\right\} + 1 - 1\end{aligned}$$

□

3.6.1 The Sweep-rule Phase In The Dual Code

Claim 3.58. *Let z be an assignment. Then:*

1. *For any generator $g \in S$, and potential wall L_g it holds that:*

$$\max_{v' \in \partial^- z} L_g(v') \geq \max_{v' \in \partial^+ \phi(z)} -L_g(v')$$

2. *The following equality holds:*

$$\max_{v' \in \partial^- z} L_{d+1}(v') = \big(\max_{v' \in \partial^+ \phi(z)} -L_{d+1}(v') \big) + d' - 1$$

Proof. For the first part, let u be a local maximum of $-L_g$ in $\partial^+ \phi(z)$. Let v be a vertex and let $S' \subset S$ be a subset of generators, of size $d'-1$, such that $\theta_{\mathbf{v},\mathbf{S'}}$ is an unsatisfied check, induced by $\phi(z)$, containing u. Let's split into cases, depending on whether $g \in S'$.

1. Assume that $g \notin S'$; thus, $L_g(v) = L_g(u)$. Since ϕ-transpose sends checks to checks, we have that the check $\partial^- \theta_{\mathbf{v^{-1}},\mathbf{S/S'}} = \phi(\partial^+ \theta_{\mathbf{v},\mathbf{S'}})$ is not satisfied, and therefore any vertex $w \in \theta_{\mathbf{v^{-1}},\mathbf{S/S'}}$ also belongs to $\partial^-(z)$. Combining that $g \notin S'$ and therefore $g \in S/S'$, we get that $gv^{-1} \in \partial^-(z)$. In particular:

$$\max_{v' \in \partial^- z} L_g(v') \geq L_g(gv^{-1}) = 1 + L_g(v^{-1}) = 1 - L_g(u)$$

2. Assume that $g \in S'$. Then $u \neq gv$ (otherwise $-L_g(u) < -L_g(v)$), and again $L_g(u) = L_g(v)$. By repeating the argument above, we get that $v^{-1} \in \partial^- z$, and therefore:

$$\max_{v' \in \partial^- z} L_g(v') \geq L_g(v^{-1}) = -L_g(u)$$

   So in overall we get that:

$$\max_{v' \in \partial^- z} L_g(v') \geq \max_{v' \in \partial^+ \phi(z)} -L_g(v')$$

For the second part, let u be a maximum of $-L_{d+1}$ in $\partial^+ \phi(z)$. Let v be a vertex and let $S' \subset S$ be a subset of generators, of size $d'-1$, such that $\theta_{\mathbf{v},\mathbf{S'}}$ is an unsatisfied check, induced by $\phi(z)$, containing u. Clearly, it must holds that:

$$u = \big(\prod_{g \in S'} g \big) v$$

Now $\theta_{\mathbf{v^{-1}},\mathbf{S/S'}}$ is an unsatisfied check in $\partial^- z$, and v^{-1} is a maximum of L_{d+1} in $\partial^- z$. To see this, assume through contradiction that the maximum of L_{d+1} in $\partial^+ z$ is different from v^{-1} and denote it by v'. Then there is an unsatisfied check containing v' where v' is its root. By moving again to the transposed image by ϕ, we find an unsatisfied check induced by $\phi(z)$ which is rooted in v', and the extremum vertex on it has a higher $-L_{d+1}$ than u.

Therefore we get the strict equality:

$$\begin{aligned} \max_{v' \in \partial^- z} L_{d+1}(v') &= L_{d+1}(v^{-1}) = -L_{d+1}(v) = -(L_{d+1}(u) - (d'-1)) \\ &= \big(\max_{v' \in \partial^+ \phi(z)} -L_{d+1}(v') \big) + d' - 1 \end{aligned}$$

□

Corollary 3.59. *Application of the Sweep rule in the dual code changes the extremum values of $\{L_g\}_{g\in S}$ and L_{d+1} exactly as it changes them in the original code. Namely, the extremum values of $\{L_g\}_{g\in S}$ don't increase, and the maximum value of L_{d+1} decreases by one.*

3.7 The bottom line

Claim 3.60. *The bottom line.* ***[COMMENT]*** *Complete it.*

3.8 Toric Encoded Circuits and Toom's Contours

Definition 3.61 (Troic Encoded Circuit)**.** *Let C be a quantum circuit. We say that C is a Toric encoded circuit if the following holds:*

1. *Any layer of the circuit consists of classical and quantum registers, such that there is a Toric code $\mathcal{C}$ such that the quantum registers are encoded by the nice encoding of $\mathcal{C}$. At time $t = 0$, the quantum registers are initialized to be either $|\bar{0}\rangle$, encoded $|\mathbf{H}\rangle$, encoded $|\mathbf{S}\rangle$, or encoded $|\mathbf{T}\rangle$. The classical registers are initialized to be zeros.*

2. *The layers of C on the quantum registers at even times are sweep-cycles, and the layers at odd times are realizations of either logical Paulis, $\mathbf{CX}$, or measurements in the computational and parity bases. On the classical registers, including the measured register, we allow any two-bit gates.*

Definition 3.62 (Base graph G_o)**.** *Let C be a Toric encoded circuit at depth d. Denote its quantum registers by $\{R_i^j\}$, where the subscript and the superscript identify the register number and the time, respectively. For example, R_0^0 is the first register at time zero. Notice that measurements take quantum registers into classical, Thus any quantum register has a maximal depth, which we denote by d_i. Let's denote by $\{\mathcal{G}_i\}$ the Toric complexes that induce the codes. We define the base graph of C, denoted by $G_o = (V_o, E_o)$, as follows:*

1. *The vertices V_o are tuples, where the first coordinates correspond to a vertex in the disjoin union over the vertices in $\{\mathcal{G}_i\}$, and the second coordinate is associated with a time. Namely:*

$$V_o := \bigcup_i \big(V(\mathcal{G}_i) \times [d_i]\big)$$

2. *The edges E_o are derived from the original edges in $\mathcal{G}_1, \ldots, \mathcal{G}_k$, with the addition that any two vertices in preceding times t and $t+1$ that support qubits (faces) such that C acts non-trivially on those qubits at time t are also connected by an edge. Namely:*

$$\begin{aligned} E_o := \big\{\{(v,t),(u,t')\} :\ & \textit{if} \\ & \textit{either there exists } i \textit{ such } \{v,u\} \in \mathcal{G}_i \textit{ and } t' = t+1 \\ & \textit{or there exists } u_i = (g_i, l_i) \in C \textit{ such } v,u \in l_{i,2} \textit{ and } t = l_{i,1}, t' = l_{i,1}+1\big\} \end{aligned}$$

[COMMENT] Here we should add that a vertex v belongs to location l if there is a qubit f that v belongs to its associated face.

Definition 3.63 (Tooms Contour Graph of $C(\mathcal{E})$)**.** *For a quantum circuit C and set of faults $\mathcal{E}$, let us denote the base graph of $C[\mathcal{E}]$ by G_o. For each time t, denote the deviation induced by $\mathcal{E}$ at time t by $\mathcal{D}_t$. We define the Tooms contour graph $\mathcal{H} = (V_\mathcal{H}, E_\mathcal{H})$ as follows:*

1. *The vertices $V_\mathcal{H}$ are the boundary of $\mathcal{D}$. Namely:*

$$V_\mathcal{H} := \{v_t : v_t = (v,t) \in V_o \text{ and there exists a face } f \in \partial\mathcal{D}_t \text{ such } v \in f\}$$

We extend the action of potential walls to act on the vertices of $V_\mathcal{H}$ by acting on the associated points in $\mathbb{R}^d$. More concretely, for a vertex $v \in V_\mathcal{H}$, whose corresponding vertex on the infinite $\mathbb{R}^d$ Toric is $\tilde{v} \in \mathcal{G}$, and a potential wall L_i, the value of L_i at v is:

$$L_i(v) := L_i(\tilde{v})$$

2. *The edges $E_\mathcal{H}$ is partitioned into two types $A_\mathcal{H}$ and $F_\mathcal{H}$, where for any logical gate g which applied in time t in $C[\mathcal{E}]$ we add edge to $A_\mathcal{H}$ as follow:*

   (a) *If g is either a logical Pauli, a measurement (both in the computational and phase basis), or an identity, then for any face f in the register on which g acts, we add:*

$$\{\{(u,t),(v,t+1)\} : u,v \in f \text{ and } v,u \in V_\mathcal{H}\}$$

   (b) *If g is a logical* **CX** *then for any physical* **CX** *in its realization, between faces f_1 into f_2 at time t we add:*

$$\{\{(u,t),(v,t+1)\} : u \in f_1 \cup f_2, v \in f_1 \cup f_2 \text{ and } v,u \in V_\mathcal{H}\}$$

   In addition for any fault g of $\mathcal{E}$, that is applied at time t, and any face $f \in Locc(g)$, we add the edges:

$$\{\{(u,t),(v,t+1)\} : u,v \in f \text{ and } u,v \in V_{\mathcal{H},X}\}$$

   To construct $F_\mathcal{H}$, we connect any pair of vertices (v,t), (u,t) if there exists a vertex $(w,t+1)$ such both of them are connected to it trough $A_\mathcal{H}$. Namely:

$$F_\mathcal{H} = \{e = \{u,v\} \in E_o : \text{ exists } w \in V_\mathcal{H} \text{ such } \{v,w\},\{u,w\} \in A_\mathcal{H}\}$$

We call $A_\mathcal{H}$ and $F_\mathcal{H}$ the arrows and forks edges respectively.

For the analyses we would like to have the following definition:

Definition 3.64. *For a Toric encoded quantum circuit C, a set of faults $\mathcal{E}$, and a step τ, according to the standard order, let $C[\mathcal{E}]_\tau$ be the partial computation circuit of $C[\mathcal{E}]$ up to step τ. We define the analysis Toom's contour at step τ $\mathcal{H}_\tau = (V_\mathcal{H}, E_\mathcal{H})$ to be the graph obtained by taking the Toom's contour of $C[\mathcal{E}]_\tau$ and adding vertices and edges as follows:*

1. *For vertices, we define $t^\star$ as the maximum time of vertices up to step τ plus one, and then add the subset:*

$$\Big\{v_{t^\star} : v_{t^\star} = (v, t^\star) \text{ and there exists a } d' \pm 1 \text{ dimensional face } f \in \partial D_{t^\star - 1} \text{ and a vertex } u_{t^\star-1} \text{ such } v_{t^\star}, u_{t^\star-1} \in f\Big\}$$

2. *For the edges, we add the following arrows:*

$$\Big\{\big\{(u, t^\star - 1), (v, t^\star)\big\} \text{ if there exists a } d' \pm 1 \text{ dimensional face } f \in \partial D_{t^\star - 1} \text{ and a vertex such } v_{t^\star}, u_{t^\star-1} \in f\Big\}$$

3. *Finally, we add forks for vertices at time $t^\star$ to preserve the property that any two vertices with the same time coordinate, which are connected to a third via arrows, are also connected by a fork.*

3.9 Space-Time Graph

Definition 3.65. *Consider a graph $G = (V, A, F)$, where V is a set of vertices, and A and F are sets of undirected edges. For $v \in V$, denote by $pre(v) = \{u \in V : \{u, v\} \in A\}$, we extend the notion for subsets of vertices $X \subset V$ by $pre(X) = \bigcup_{v \in X} pre(v)$. Let $T : V \to \mathbb{N}$, We say that (G, T) is a time graph if and only if:*

1. *$\{x, y\} \in A \Rightarrow T(y) = T(x) + 1$*
2. *For $x, y \in V$ $\{x, y\} \in F$ if and only if there exists $z \in V$ such both $\{x, z\}, \{y, z\} \in A$.*

We will call to T time-function, to A arrows, and to F forks. Observes that from the second requirement it follows that forks can connect only vertices that have the same time value (set by T).

Remark 3.66. *The Toom's contour graph defined in Definition 3.63 is, by definition of it's construction, a time-graph.*

Definition 3.67 (History, slice.)**.** *For a time graph (G, T) and a vertex $u \in V$, let G_u be the subgraph of (V, A) induced by $\{v \in V : T(v) \leq T(u)\}$. We will denote by H_u the 'History of u', which is the set of vertices that are connected to u in G_u. A slice is an equivalence class of the relation $u \equiv v$ if and only if $H_u = H_v$. Notice that the time function has to be constant on a slice, to see this consider vertices x, y with $T(x) > T(y)$, thus $x \notin G_y$ and therefore $x \in H_x / H_y$ So they are belong to different classes. Thus, we can extend the time function and the history to slices and write for a slice K, $T(K)$ and H_K.*

Claim 3.68. *Consider slices K_1, K_2 such that $T(K_1) = T(K_2)$, then either $K_1 = K_2$ or H_{K_1}, H_{K_2} are disjoint.*

Proof. Assume a non-empty intersection, so there exists $z \in H_{K_1} \cap H_{K_2}$ such that for any $x \in K_1$ and $y \in K_2$, there exist arrow-paths $z \to x$ and $z \to y$. Thus, for any $w \in H_{K_1}$, one can concatenate a path $w \to x \to z \to y$ and conclude that $y \in H_{K_2}$. □

Definition 3.69 (The graph of slice)**.** *Let K be a slice. The graph of K, denoted by $G_K = (V_K, E_K)$, is the graph whose vertices are slices $R \subset H_K$ with $T(R) = T(K) - 1$. Two vertices are connected by an edge if and only if there is a fork whose ends belong to the slices associated with them. Namely, $S, R \in V_K$ are connected in G_K if and only if there is $\{s, r\} = f \in F$ such that $s \in S$ and $r \in R$.*

Claim 3.70. *G_K is connected.*

Proof. Let $R, S \in V_K$ and consider $r, s \in V$ such that $r \in R, s \in S$. Recall that $R, S \subset H_K$ and therefore $r, s \in H_K$, Thus, by definition of the 'History' there exists a path, passing trough arrows only, from r to s in H_K. Consider such a path that is minimal in the number of points x belong to it, with $T(x) = T(K)$. By induction on the number k of such points we prove that R and S are connected in G_K.

1. Base. $k = 0$, In this case, the path from r to s lies within $G_r (= G_s)$. Hence $H_r = H_s$, So $R = S$.

2. Induction step. There exists y on the path from r to s such that $T(y) = T(K)$. Let x be the point which precedes y on the path, and z the point that follows y. Since the values of T at the two ends of an arrow differ by 1, and since y gets the maximal T value in H_K it follows that:

   (a) $\{x, z\} \subset \mathrm{pre}(y) \Rightarrow \{x, z\} \in F$.

   (b) The slices of x and z, say K_x and K_z has time value $T(K_x) = T(K_z) = T(K) - 1$, Namely they are both members of V_K.

   Since the path connecting r, x in H_K has $k - 1$ vertices with $T(\cdot) = T(K)$, then by the induction hypothesis, R and K_x are connected in G_K, and by similar argument so are K_z and S. On the other hand, $\{x, z\} \Rightarrow \{K_x, K_z\} \in E_K$. Thus, R and S are connected in G_K.

□

Definition 3.71 (Spanned Set)**.** *Consider a subset $A \subset \mathbb{R}^d$, and $d + 1$ points $x_1, \ldots, x_{d+1}$. We say that $\langle A, x_1, \ldots, x_{d+1} \rangle$ is a spanned set of A if for any $i \in [d+1]$ and $s \in A$, it holds that $L(s) \leq L_i(x_i)$ and in addition there are $v_1, ..v_{d+1} \in V$, not necessary different, that achieve the equalities $L_i(x_i) = L_i(x_i)$. We name $x_1, \ldots, x_{d+1}$ the poles of A. Furthermore, we extend the notation to a spanned slice when A is a slice of some time-graph. Notice that saying $\langle A, x_1, \ldots, x_{d+1} \rangle$ is a spanned set means that* $\mathbf{Size}(A) = \mathbf{Size}(\{x_1, \ldots, x_{d+1}\})$.

Observe that in the case where the subset A is a slice. Each of these poles can be associated with a vertex of G that gives the equality. If there are multiple vertices whose give equality, associate the point with one of them arbitrarily. We will use the notion of applying functions originally defined over vertices to the poles, referring to the application over the vertices associated with them. For example $\text{pre}(x_i) = \text{pre}(v)$ for the vertex $v \in A$ which satisfies $L_i(v) = L_i(x_i)$ and therefore is associated with the pole x_i. Another example is referring to x_i as a vertex, so when saying 'For $x_i \in V_k$' or 'For $\{x_i, u\} \in F$', we refer to the associated vertex or the edge connecting it to u.

Claim 3.72. *Consider a time graph (G, T), Let K be a slice in G, and let $\hat{K} = \langle K, x_1, ..x_{d+1}\rangle$ be a spanned slice of K such that $K \neq H_K$. Then for some integer k there exist spanned slices $\hat{K}_0 = \langle K_0, \cdot\rangle, .., \hat{K}_k = \langle K_k, \cdot\rangle$ and Forks $F_1, .., F_k$ such that:*

1. *For $i \in [d+1]$ $pre_i(x_i)$ belongs to the poles $\hat{K}_0, .., \hat{K}_k$.*
2. *Introduce an edge between $\hat{K}_i$ and $\hat{K}_j$ if and only if one of the Forks $F_1, .., F_k$ consists of a pole of $\hat{K}_i$ and a pole of $\hat{K}_j$. Then the graph formed from $\hat{K}_0, .., \hat{K}_k$ is connected.*
3. $\sum_{i=0}^{k} \textbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \textbf{Size}(F_i) \geq \sum_{i=1}^{d+1} L_i\left(pre_i(v_i)\right)$

Proof. First Notice that K doesn't contain leaves in (V, A), since a leaf is connected only to itself via arrows, and therefore its 'slice' equals to its 'History'. Thus $\text{pre}_i(x_i)$ is not empty for $i = 1, .., d+1$. Next consider the graph $G_K = (V_K, E_K)$ from definition 3.69. Since $\{x_i, \text{pre}_i(x_i)\}$ is an arrow $\text{pre}_i(x_i)$ belongs to a slice in V_K. for $i = 1, .., d+1$ denote by R_i the slice in V_K that contains $\text{pre}_i(x_i)$.

By claim 3.70 G_K is connected and therefore there exist a minimal collection $\{K_0, .., K_k\}$ of slices form V_K which contains $R_1, R_2, ..R_{d+1}$, and which is connected in G_K. We pick a set of forks $\mathbf{F} = \{F_1, .., F_k\}$ which realizes a spanning tree for this collection of slices. Later we will identify edges from this spanning tree with the forks from $\mathbf{F}$.

For $i = 1, .., d+1$ we set $\text{pre}_i(v_i)$ to be the ith pole of R_i, This assures (1). The set of remaining poles for $\hat{K}_0, .., \hat{K}_k$ will be equal to $\cup_{i=k}^{k} F_i$. This will assure (ii), We will also select poles for $F_1, .., F_k$ to form 'spanned forks' $\hat{F}_1, \cdots, \hat{F}_k$ in such way that:

$$\sum_{i=0}^{k} \textbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \textbf{Size}(\hat{F}_i) \geq \sum_{i=1}^{d+1} L_i\left(\text{pre}_i(v_i)\right)$$

This will assure (3).

Let us denote by $J_0, .., J_{2k}$ the enumerated union $K_0, ..K_k, F_1, .., F_k$. Observes that for any choice of $(d+1)(2k+1)$ elements $z_0^1, z_0^2, z_0^3, .., z_0^{d+1} .., z_{2k}^{d+1}$ in $\mathbb{R}^d$, such that $z_j^i \in J_j$ it holds that:

$$\sum_{i=0}^{k} \textbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \textbf{Size}(\hat{F}_i) \geq \sum_{j,i} L_i(z_j^i) \tag{1}$$

Later, we are going to take the points $\{z_j^i\}$ from the intersection of J_j's. For convenience, let us assume that for the index subset $X \subset [2k] \cup \{0\}$, the intersection $\bigcap_{j\in X} J_j$ returns a single point. If there is more than one, then decide on the point in a way that all the chosen points simultaneously satisfy that for any index subsets Y, X, such that $X \cap Y$ and they both introduce a non-trivial intersection $\bigcap_{j\in Z} J_j : Z \in \{X, Y\}$, then:

$$\bigcap_{j\in X} J_j = \bigcap_{j\in Y} J_j$$

Now consider a non-trivial maximal intersection $\bigcap_{j\in X} J_j$, the intersection is maximal in the sense that for any $j' \notin X$ it holds that $.\bigcap_{j\in X\cup\{j'\}} J_j = \emptyset$. We claim that for any i and for any such X, there is $t \in X$ such that J_t is the successor of J_j on the path to R_i for any $j \in X/\{t\}$. Assume not. Let $x, y \in X$, and denote the paths from J_x, J_y to the ith pole by $\langle J_x, J_{x_2}, \ldots, R_i\rangle$, $\langle J_y, J_{y_2}, \ldots, R_i\rangle$, such that $J_y \neq J_{x_2}$. If $y = x_2$, $y_2 = x$, or $x_2 = y_2 \in X$, then pick other x, y (if there is no such, we get an immediate contradiction). In that case, $\langle J_x, J_y, J_{y_2}, \ldots, R_i\rangle$ is a path from J_x to R_i which is different from $\langle J_x, J_{x_2}, \ldots, R_i\rangle$. Therefore, there are at least two paths from J_x to the ith pole, namely, there is a cycle in contradiction to the minimality of $K_0, \ldots, K_k$.

We will choose the z_j^i by the following process. Pick a z_j^i that has not been set yet. If $J_j = R_i$, then set $z_j^i \leftarrow \mathrm{pre}_i(x_i)$ - we call this a type-1 assignment. Otherwise, consider the path from J_j to R_i (by connectivity, it has to be such a one, and by minimality, there is exactly one). Let J_t be the successor on the path, then pick $z_j^i \in J_j \cap J_t$ - similarly, we call this assignment a type-2 assignment. Since any type-2 assignment is associated with a non-trivial intersection, and any non-trivial intersection induces a type-2 assignment for all i's (since for all i's, one of the J_j on its support is a successor of the rest on their path to the ith pole), we have that:

$$\begin{aligned}
\sum_{j,i} L_i(z_j^i) &= \sum_{j,i,z_j^i\in\text{type-1}} L_i(z_j^i) + \sum_{j,i,z_j^i\in\text{type-2}} L_i(z_j^i) \\
&= \sum_{i}^{d+1} L_i\left(\mathrm{pre}_i(x_i)\right) + \sum_{X\colon \bigcap_{j\in X} J_j\neq\emptyset} (|X|-1) \sum_{i}^{d+1} L_i\left(\bigcap_{j\in X} J_j\right) \\
&= \sum_{i}^{d+1} L_i\left(\mathrm{pre}_i(x_i)\right)
\end{aligned}$$

Plugging it into eq. (1) gives the desired. □

Claim 3.73. *The maximal size of a fork is $2d' = d$.*

Proof. Consider the rules of forks connecting in Definition 3.63. For forks obtained by rules (a), (b), or as a result of a fault, it's clear that their vertices at their ends are on intersecting faces (when projecting the vertices on $\mathcal{G}$). The maximal fork, in those cases, has the form of:

$$F = \{v, (\prod_{g\in S_1'} g)(\prod_{g\in S_2'} g)v\}$$

Where S'_1 and S'_2 are subsets of S of size d', and therefore the size of such forks is $\mathbf{Size}(F) = 2d'$. □

Definition 3.74. *Let $\hat{K}$ be a spanned slice, and let $\hat{K}_0, \ldots, \hat{K}_k$ be spanned slices in H_K, and let $F_1, \ldots, F_k$ be spanned forks, such that $\hat{K}_0, \ldots, \hat{K}_k, F_1, \ldots, F_k$ satisfy the conditions in Claim 3.72. We call $\hat{K}_0, \ldots, \hat{K}_k, F_1, \ldots, F_k$ the predecessor decomposition of $\hat{K}$.*

Furthermore, we say that $\hat{K}$ is a result of a sweep-cycle, faults, or a logical operation if the layer at time $T(\hat{K}) - 1$ consists of sweep-cycle gates, fault gates (whose source is $\mathcal{E}$), or either measurement or logical Pauli. The order of the slice is its time modulo 4, namely $\mathrm{order}(\hat{K}) = T(\hat{K}) \mod 4$. Observe that slices that are results of faults have an order that is either 0 or 2, whereas slices that are results of a sweep-cycle or logical operation are of orders 1 and 3, respectively.

Corollary 3.75. *Consider a spanned slice $\hat{K}$, at finite size, such that $K \neq H_K$. Since $K \neq H_K$, it follows that $\hat{K}$ has a predecessor decomposition, denoted by $\hat{K}_0, \ldots, \hat{K}_k, F_1, \ldots, F_k$. The inequality in Claim 3.72 becomes:*

1. *If $\hat{K}$ is a result of ξ-sweep-cycle, then:*

$$\sum_{i=0}^{k} \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K}) + \xi$$

2. *If $\hat{K}$ is a result of either faults or a logical operation, then:*

$$\sum_{i=0}^{k} \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K})$$

Proof. For the sweep-cycle part, we use Claim 3.60, which states that after each application of the sweep-cycle over a finite size assignment, the maximal L_{d+1} value of the resulting boundary decreases by 1, while the values of each L_g do not increase.

When $\hat{K}$ is a result of either faults[7] or a logical operator, we use the fact that since the operators are transversal, we have that the support of $\hat{K}$ is contained in the support of $\cup_0^k \hat{K}_i$, and therefore for any $i \in [d+1]$:

$$\max\{L_i(v) : v \in \hat{K}\} \leq \max\{L_i(v) : v \in \bigcup_{i=0}^{k} \hat{K}\}$$

□

Claim 3.76. *For a step τ, let $\mathcal{H}_\tau$ be analysis Toom's contour at step τ, and let $t^\star$ be the maximal time at that contour. For a spanned slice $\hat{K}$, at time $T(K) = t^\star$, where $K \neq H_K$, the inequality in Claim 3.72 becomes:*

$$\sum_{i=0}^{k} \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \mathbf{Size}(F_i) \geq \mathbf{Size}(K) - d - d' - 1$$

[7] Notice that $\hat{K}$ does not consist of faults since otherwise $K = H_K$, so the meaning here is that $\hat{K}$ contains vertices which weren't flipped in the fault layer.

Proof. We allow vertices at the final time layer to be connected by an arrow if they are at a distance of $d'+1$ from each other. Therefore, for any $i \in [d]$, the value of L_i for v_i might be greater by 1 than the value of $\mathrm{pre}_i(v_i)$, and the L_{d+1} for v_{d+1} might be greater by $d'+1$ than the value of the $d+1$ pole of $\hat{K}$. Hence, overall, we get that:

$$\begin{aligned}\sum_{i=1}^{d+1} L_i\left(\mathrm{pre}_i(v_i)\right) &\geq \sum_{i=1}^{d}\big(L_i\left(v_i\right)-1\big) + L_{d+1}(v_{d+1}) - \max d' - 1 \\ &= \mathbf{Size}(K) - d - d' - 1\end{aligned}$$

□

3.10 Explantation of a Spanned Slice

Definition 3.77. *Consider a Tooms contour graph induced by Toric encoded circuit, with sweep-cycle parameter $\xi = 1$, and let α, β be constants and $\mathcal{X}_0, \mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3 \in \mathbb{A}$ be quadruple, defining such:*

$$\alpha := 40(d+1)^2, \quad \beta := 5(d+1)$$
$$\mathcal{X}_r := 1 - r\frac{d+1}{\alpha} = 1 - \frac{r}{40(d+1)}$$

Given sets W of points, A of arrows and F of forks, we define U as the set of vertices of the graph induced by the edges of $A \cup F$. The sets W, A and F form an explanation of a spanned slice $\hat{K} = \langle K, v_1, .., v_{d+1}\rangle$ of order $\mathrm{order}(\hat{K}) = r$, if and only if:

1. *$W \cup \{v_1, .., v_{d+1}\} \subset U \subset H_K$ and the graph $(U, A \cup F)$ is connected.*
2. *All elements of W are leaves.*
3. *For $m = |W|$ and $s = \mathbf{Size}(K)$ we have $|A| \leq \alpha(m - \mathcal{X}_r) - \beta s$ and $|F| < m - 1$.*

Furthermore, for a step τ and a spanned slice $\hat{K}$ in the analysis Toom's contour at step τ, we extend the definition of explanation as above, but require the following weaker inequality instead: For $m = |W|$ and $s = \mathbf{Size}(K)$, we have $|A| \leq \alpha(m+1) - \beta s$ and $|F| < m - 1$.

Claim 3.78. *Every spanned slice $K = \langle K, v_1, .., v_{d+1}\rangle$ has an explanation.*

Proof. Let $h = T(K) - \min_{u \in H_K} T(u)$. For spans derived from the standard Toom's contours (namely not a analysis Toom's contours), we prove by induction on h.

1. Base: $h = 0$, meaning T is constant on H_K, so no arrows connect points of H_K. Thus, H_K, as well as $\mathbf{K}$, consists only of a single leaf, namely $\hat{K}$ is a result of a fault, and $r = \mathrm{order}(\mathcal{X}_r) \in \{0, 2\}$. In that case, there is a single vertex $u \in V_{\mathcal{H}}$ such that $U = W = \{u\}$, so $m = 1$, $s = 0$, and $|A|, |F| = 0$. It is easy to check that those values satisfy the requirement.

2. Induction Step: $h \neq 0$ implies $K \neq H_K$. Thus, by claim 3.72, we know that for some k there exist spanned slices $\hat{K}_0, .., \hat{K}_k$ and forks $F_1, .., F_k$ that satisfy claim 3.72. In particular, for $r < 3$ it holds that:

$$\sum_{i=0}^{k} \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K})$$

Whereas, for $r = 3$:

$$\sum_{i=0}^{k} \mathbf{Size}(\hat{K}_i) + \sum_{i=1}^{k} \mathbf{Size}(F_i) \geq \mathbf{Size}(\hat{K}) + \xi = \mathbf{Size}(\hat{K}) + 1$$

Consider $i \in \{0, \ldots, k\}$, and denote $s_i = \mathbf{Size}(\hat{K}_i)$. By the inductive hypothesis and since $T(K_i) = T(K) - 1$, we know that $\hat{K}_i$ has an explanation formed by a set of leaves $\mathbf{W}_i$, a set of arrows $\mathbf{A}_i$, and a set of forks $\mathbf{F}_i$. Let $m_i = |\mathbf{W}_i|$. Thus, the sets $\mathbf{A}_i$ and $\mathbf{F}_i$ have at most $\alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i$ arrows and $m_i - 1$ forks. We define:

(a) $\mathbf{W} = \cup_{i=0}^{k} \mathbf{W}_i$, Notice that $\mathbf{W}_i \subset K_i$ and that $T(K_i) = T(K_j)$ for all i, j, thus by Claim 3.68 $\mathbf{W}$ is a union of disjoint set, Hence W contains $m = \sum_{i=0}^{k} m_i$ leaves.

(b) $\mathbf{F} = \{F_1, .., F_k\} \cup_{i=0}^{k} \mathbf{F}_i$ with at most $k + \sum_{i=0}^{k} m_i - 1 = m - 1$ elements.

(c) $\mathbf{A} = \{\{v_i, \mathrm{pre}_i(v_i)\}\} \cup_{i=0}^{k} \mathbf{A}_i$. For bounding the number of arrows in $\mathbf{A}$ we condition upon whether $r = 3$, For convenient, let us denote by Y the indicator, which indicate if $r = 3$.

$$\begin{aligned}
&d + 1 + \sum_{i=0}^{k} \alpha(m_i - \mathcal{X}_{r-1}) - \beta s_i \\
\leq &d + 1 + \alpha m - \mathcal{X}_{r-1}\alpha(k+1) - \beta\left(s + Y\xi - k \cdot \mathbf{Size}(\text{fork})\right) \\
= &\alpha\left(m - \mathcal{X}_r\right) - \beta s + \left(\alpha(\mathcal{X}_r - \mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1}k\alpha - \beta Y\xi + \beta k \cdot \mathbf{Size}(\text{fork})\right)
\end{aligned}$$

So it's left to show that the term in the closure is negative for all integers k:

$$\alpha(\mathcal{X}_r - \mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1}k\alpha - \beta Y\xi + \beta k \cdot \mathbf{Size}(\text{fork})$$

We do that by showing that for the choice of the parameter values, $\alpha, \beta, \mathcal{X}_0, \ldots, \mathcal{X}_3$, the slope is negative, and then the value of the term at k equals zero is negative. For the slope to be negative, we require:

$$-\alpha\mathcal{X}_{r-1} + \beta\mathbf{Size}(\text{fork}) \leq 0 \Rightarrow \mathbf{Size}(\text{fork}) \leq \frac{\alpha}{\beta}\mathcal{X}_{r-1}$$

For negativity at the origin, $k = 0$, we require for $r < 3$:

$$\begin{aligned}
(\mathcal{X}_r - \mathcal{X}_{r-1})\alpha + d + 1 \leq 0 \\
\Rightarrow \mathcal{X}_r \leq \mathcal{X}_{r-1} - \frac{d+1}{\alpha}
\end{aligned}$$

And for $r = 3$:

$$(\mathcal{X}_3 - \mathcal{X}_2)\alpha + d + 1 - \beta\xi \leq 0$$
$$\Rightarrow \mathcal{X}_3 \leq \mathcal{X}_0 - 3\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi$$

So in total:

$$\mathcal{X}_1 \leq \mathcal{X}_0 - \frac{d+1}{\alpha}$$
$$\mathcal{X}_2 \leq \mathcal{X}_0 - 2\frac{d+1}{\alpha}$$
$$\mathcal{X}_3 \leq \mathcal{X}_0 - 3\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi$$
$$\mathcal{X}_0 \leq \mathcal{X}_0 - 4\frac{d+1}{\alpha} + \frac{\beta}{\alpha}\xi$$
$$\textbf{Size}(\text{fork}) \leq \frac{\alpha}{\beta}\mathcal{X}_r$$

The first three inequalities hold by the definitions of the $\mathcal{X}_r$, the fourth is satisfied since $\frac{\beta}{\alpha}\xi = 4\frac{d+1}{\alpha}$, the last is obtained due to Claim 3.73, that states that the maximal size of a fork equals $4d$, and that $\frac{\alpha}{\beta}\mathcal{X}_r$ is greater than $4d$.

Finally, by Claim 3.72, the graphs induced by $\mathbf{A}_i \cup \mathbf{F}_i$ are connected, thus the graph induced by $\mathbf{A} \cup \mathbf{F}$ is also connected. Therefore, $\mathbf{W}, \mathbf{A}, \mathbf{F}$ form an explanation of K.

It's left to prove for spanned slices in the analysis Toom's contours. Notice, that h has to be greater than zero, We prove the inequality by repeating on the induction bases, where all the spanned slices that are back in the history of K are spanned slices in the standard Toom's contour. So the only different is in the bounding of $\mathbf{A}$'s size. We use Claim 3.76 and get:

$$\begin{aligned}|\mathbf{A}| &\leq \alpha\,(m+1) - \beta s \\ &+ (-\alpha(1+\mathcal{X}_{r-1}) + d + 1 - \mathcal{X}_{r-1}k\alpha + d + d' + 1 + \beta k \cdot \textbf{Size}(\text{fork})) \\ &\leq \alpha\,(m+1) - \beta s \\ &+ (-\alpha + d + 1 - \mathcal{X}_{r-1}k\alpha + d + d' + 1 + \beta k \cdot \textbf{Size}(\text{fork}))\end{aligned}$$

Where we used the fact that $\mathcal{X}_{r-1} \in (0,1)$ for $r \in \{0,1,2,3\}$. Clearly, the bound is satisfied since $\alpha \geq 2d + d' + 1$ and $\mathcal{X}_{r-1}\alpha \geq \beta \cdot \textbf{Size}(\text{fork})$. □

Corollary 3.79. *Let $\hat{K}$ be a spanned slice with size s, and explanation with $|A|$ arrows, and $|F|$ forks. Then it is induced by at least the following number of faults:*

$$\frac{1}{2^{d'}}\frac{1}{2}\left(|F| + 1 + \alpha^{-1}(|A| - 1) + \beta s\right)$$

Furthermore, the ratio of leaves to edges in an explanation is greater than $\frac{1}{2\alpha} = \frac{1}{80(d+1)^2}$.

Proof. We bound the Claim 3.78 by the number of leaves in the explanation, and use the fact that any d'-face has $2^{d'}$ vertices, so each fault contributes at most $2^{d'}$ leaves to the explanation. □

[**COMMENT**] Check what happened if we set $\xi = \sqrt{d}, \beta = \sqrt{d}$ is that better?

3.11 Counting Subtrees.

Claim 3.80. *Let $G = (V, E)$ be a $\Delta + 1$-regular graph, and let k be an integer less than $|E|$. Fix a vertex $r \in V$. The number of subgraphs rooted at r with less than k edges is at most α^k.*

Proof. Denote by T_Δ and T_2 the infinity Δ and binary tress. There is an injection between the subtrees of G, rooted at v, with at most k edges, to the subtrees of T_Δ rooted at v, with at most k edges. Moreover, there is an injection from the subtrees of T_Δ rooted at v, with at most k edges to the subtrees of T_2, rooted at v, with at most $k \log \Delta$ edges.

[**COMMENT**] In the second map, we think on replacing each of the vertex with $\Delta - 1$-leaves binary tree, each leaf is associated with a outgoing edge. .

Thus, it's enough to bound the number of of subtrees in T_2 rooted at v, with at most $k \log \Delta$ edges, that number is less than the number of binary tress with $k \log \Delta + 1$ leaves. For exact leaves number we get the $k \log \Delta$ Catalan number, which its limit is:

$$\sim \frac{4^{k \log \Delta}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{(k \log \Delta)^{\frac{3}{2}} \sqrt{\pi}}$$

Therefore we can bound by:

$$\sum_{j=1}^{k} \frac{\Delta^{2j}}{(j \log \Delta)^{\frac{3}{2}} \sqrt{\pi}} \leq \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta} \sum_{j=1}^{k} \frac{1}{j^{\frac{3}{2}}} \leq \zeta(\frac{3}{2}) \cdot \frac{\Delta^{2k}}{\sqrt{\pi} \log^{\frac{3}{2}} \Delta}$$

□

3.12 Tree Separator Lemma

For a tree $T = (V, E)$ and a vertex $v \in V$, we call the subtree obtained by taking the descendants of v the subtree rooted at v.

Claim 3.81. *Let $T = (V, E)$ be a tree with $m > 3$ leaves and τ edges, then it has a subtree with $m' \in (\frac{1}{3}m, \frac{2}{3}m)$ leaves and at most $\frac{m'}{m}\tau$ edges.*

Proof. Order the vertices according to their heights. Denote by $v \in V$ the lowest vertex for which the subtree rooted at v has at least $\frac{1}{3}m$ leaves. Now, denote by T_1 the subtree obtained through the following iterative process over v's children:

1. Initialize T_1 as the subtree rooted at v.
2. For a child u of v:

(a) Check if erasing from T_1 the subtree rooted at u doesn't decrease the number of leaves below $\frac{1}{3}m$. If so, then we update T_1 to be the result of the erasure.

Clearly, the algorithm is defined such that T_1 has to have at least $\frac{m}{3}$ leaves. Furthermore, since any child of v is a root of a subtree with fewer than $\frac{m}{3}$ leaves (otherwise we get a contradiction to the minimality of v's height), we have that if T_1 would have more than $\frac{2}{3}m$ leaves, then the subtraction of any of its children would yield a tree with at least $\frac{2}{3}m - \frac{1}{3}m = \frac{1}{3}m$ leaves. Namely, v has in T_1 a child u such that the subtraction of the subtree rooted at u from T_1 results in a subtree with more than $\frac{1}{3}m$ leaves. But if indeed there were such a child, then the algorithm would erase it, so there is not.

Now let $T_2 = T/T_1 \cup v$, namly the tree induced by the subtraction of T_1's edges from T. Observes that since the summation of leaves in T_1 and T_2 is the leaves in T, and T_1 has no more than $\frac{2}{3}m$ leaves, and no less than $\frac{1}{3}$ leaves, it holds that the number of leaves in T_2 is also in the range $(\frac{1}{3}m, \frac{2}{3}m)$. We claim that at least one of the subtrees has fewer than $\frac{m'}{m}\tau$ edges.

To see this, denote by m_1, m_2 and by τ_1, τ_2 the leaves and the edges in T_1, T_2 respectively. Now:

$$\frac{\tau}{m} = \frac{\tau_1 + \tau_2}{m_1 + m_2} = \frac{m_1}{m_1 + m_2}\frac{\tau_1}{m_1} + \frac{m_2}{m_1 + m_2}\frac{\tau_2}{m_2}$$

Namely, $\frac{\tau}{m}$ is a weighted average of $\frac{\tau_1}{m_1}$ and $\frac{\tau_2}{m_2}$, and therefore it is greater than their minimum. Peak the subtree with the minimal quotient. □

By repeating recursively on the above construction, we get the following corollary:

Corollary 3.82. *Let $T = (V, E)$ be a tree with m leaves and τ edges. Then, for any $\alpha > 0$, it has a subtree with $m' \in \left(\frac{1}{3}\alpha m, \frac{2}{3}\alpha m\right)$ leaves and at most $\frac{m'}{m}\tau$ edges.*

Claim 3.83. *Let $\gamma, \beta \in \mathbb{R}_{>0}$ and n be a positive integer. For any tree T with $\tau \geq \beta n$ edges and m leaves such that $\tau \leq \gamma m$, there is a subtree T' of T with a number of edges τ' less than $\frac{\beta}{2}n$ and a number of leaves greater than $\frac{\beta}{4\gamma}n$.*

Proof. Since the construction of claim 3.81 preserves the ratio of edges to leaves, we have that for a T' obtained using the construction, the number of edges satisfies $\tau' \leq \gamma m'$. So, it's enough to look for the maximal α such that:

$$\gamma m' \leq \gamma\frac{2}{3}\alpha m \leq \frac{\beta}{2}n \Rightarrow \alpha = \frac{3\beta}{4\gamma}\frac{n}{m}$$

Which gives $m' \geq \frac{1}{3}\alpha m = \frac{\beta}{4\gamma}n$.

□

3.13 Proof of The Theorem

Definition 3.84 (Simulation and Logical Errors)**.** *For a quantum code $\mathcal{C}$ of length n and a decoder for $\mathcal{C}$, denoted by D, we say that a Pauli operator $P \in \mathcal{P}^{\otimes n}$ is a logical error with respect to D if DP does not equal the logical identity.*

For quantum circuits C and $\tilde{C}$, a noise channel $\mathcal{E}$, and $p \in (0,1)$, we say that $\tilde{C}$ simulates C with probability $1-p$ if

1. *If there is a mapping between the qubits of C and the registers of $\tilde{C}$, along with a decoding procedure such that, at the end of running $\tilde{C}$, decoding the registers results in the same quantum states as those produced by running C.*

2. *The probability induced by $\mathcal{E}$ that the deviation on one of the registers has a logical error is less than p.*

Lemma 3.85. *For any d, d', and a constant $p_0 \in (0,1)$ that depends only on d. Consider a quantum circuit C with depth μ and width w, and let $\mathcal{N}_\mathcal{E}$ be a local stochastic noise channel with a noise rate $p < p_0$. There is a Toric encoded circuit $\tilde{C}$ such that each of its quantum registers is encoded by an n-length, (d,d')-Toric code. It simulates C with probability:*

Proof. We construct $\tilde{C}$ as follows: for each input wire of C, we initialize the logical zero $|\mathbf{0}\rangle$ of the (d,d')-Toric code. For each of the gates **H** and **S**, we initialize the encoded $|\mathbf{H}\rangle$ and $|\mathbf{S}\rangle$ states, and for each **T**, we initialize copies of both $|\mathbf{T}\rangle$ and $|\mathbf{S}\rangle$ states. The even layers of $\tilde{C}$ are sweep cycles, while on the odd layers, we set the realization of the logical gates. For **CX**, we set the transversal realization, and for **H**, **S**, and **T**, we use gate teleportation. Thus the width of $\tilde{C}$ expands to $wn + 2\mu n$. For gate teleportation, we use the decoder in **??**, which runs at most at a depth of n. Thus, the depth of $\tilde{C}$ expands to $2(3\mu n)$.

The decoder successfully decodes any error that does not contain a deviation cluster larger than $\gamma n^{\frac{1}{d}}$. Thus, it remains to prove that with probability $1-p$, no deviation cluster exceeds that size. To prove this, we use Claim 3.15, which states that the probability of there being a time t with an error cluster larger than $\gamma n^{\frac{1}{d}}$ is bounded by the number of locations multiplied by the probability that step τ is the first to admit a deviation cluster of size $\gamma' n^{\frac{1}{d}}$, maximized over τ. This leads us to prove the following claim:

Claim 3.86. *There exist constants $c_3, c_4, c_5, c_6, \alpha \in \mathbb{R}^+$, such that for ant integer s and for any step τ in the standard order of $\tilde{C}[\mathcal{E}]$, the probability that τ is the first step where there is a deviation cluster of size greater than s is less than:*

$$c_3(w+2\mu)\mu n^2 \left(\alpha p^{c_4}\right)^s + c_5((w+2\mu)\mu n^2)^2 \left(\alpha p^{c_6}\right)^{\frac{1}{2d} n^{\frac{1}{d}}}$$

Proof. Denote by B_τ the event where τ is the first step to have a deviation cluster of size greater than $\frac{1}{2} n^{\frac{d'}{d}}$, and by A_τ the event where all explanations of deviation clusters up to and including step τ have fewer than $\frac{1}{d} n^{\frac{1}{d}}$ edges. Thus:

$$\mathbf{Pr}_{\sim \mathcal{N}_\mathcal{E}}\left[B_\tau\right] \leq \mathbf{Pr}_{\sim \mathcal{N}_\mathcal{E}}\left[B_\tau | A_\tau\right] + \mathbf{Pr}_{\sim \mathcal{N}_\mathcal{E}}\left[\bar{A}_\tau\right]$$

We begin by bounding the contribution of the right term. According to Corollary 3.79, there exists a constant c_1, depending only on d, such that the ratio of leaves to edges in the explanation is at least c_1. By Claim 3.83, it follows that any such explanation has

more than $\frac{1}{d}n^{\frac{1}{d}}$ edges containing a subtree with fewer than $\frac{1}{2d}n^{\frac{1}{d}}$ edges, with a leaves-to-edges ratio greater than $\frac{1}{2}c_1$. Using the union bound, we bound the probability that there is a vertex in the base graph that is a root of such a subtree.

$$\mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\left[\bar{A}_\tau\right] \leq \bigcup_{v\in V_o,\tau'\leq\tau} \mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\Big[v \text{ is a root of subtree in } \mathcal{H}_{\tau'} \text{ with less than } \frac{1}{2d}n^{\frac{1}{d}} \text{ edges and more than } \frac{1}{2}c_1\frac{1}{2d}n^{\frac{1}{d}} \text{ leaves}\Big]$$

Now, since any two connected vertices are at a space distance of at most

$$\max\{\mathbf{Size}(\text{arrow}), \mathbf{Size}(\text{fork})\} \leq d' \leq d$$

It follows that all the vertices in the subtree are at a space distance of at most $n^{\frac{1}{d}}$ from each other. Thus, no two of them have the same space coordinate modulo n. In particular, projecting the infinite plane back to the finite Toric sends each of the leaves of the subtree to a unique target, and therefore the number of faults that induce the explanation is at least:

$$\frac{1}{2^{d'}}\frac{1}{4d}c_1 n^{\frac{1}{d}}$$

Plugging into the union bounds yields

$$\mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\left[\bar{A}_\tau\right] \leq \tau\cdot 3(w+2\mu)6\mu n^2\alpha^{\frac{1}{2d}n^{\frac{1}{d}}}p^{\frac{1}{2^{d'}}\frac{1}{4d}c_1 n^{\frac{1}{d}}} \leq ((w+2\mu)6\mu n^2)^2\left(\alpha p^{\frac{1}{2^{d'+1}}c_1}\right)^{\frac{1}{2d}n^{\frac{1}{d}}}$$

It remains to bound the term. By Claim 3.78, there is a constant c_2 depending only on d, such that any explanation of a slice of size s has at least $c_2 s$ edges. Therefore, it suffices to bound the probability that the 'analysis Toom's contour', $\mathcal{H}_\tau$ admits an explanation with more than $c_2 s$ edges. Since the event is conditioned on the event A_τ, no explanation has more than $\frac{1}{d}n^{\frac{1}{d}}$ edges. Thus, the same argument as above shows that the projection of each explanation tree into the finite toric sends its leaves to unique targets. Hence, we obtain:

$$\begin{aligned}\mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\left[B_\tau|A_\tau\right] &\leq \bigcup_{v\in V_o,x\geq c_2 s} \mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\left[v \text{ is a root of explanation with number of edges } x|A_\tau\right] \\ &\leq \sum_{x=c_2 s}^{\infty}(w+2\mu)6\mu n^2\left(\alpha p^{\frac{1}{2^{d'}}c_2}\right)^x = (w+2\mu)6\mu n^2\frac{\left(\alpha p^{\frac{1}{2^{d'}}c_2}\right)^s}{1-\left(\alpha p^{\frac{1}{2^{d'}}c_2}\right)}\end{aligned}$$

Combing the bounds we have:

$$\mathbf{Pr}_{\sim\mathcal{N}_\mathcal{E}}\left[B_\tau\right] \leq (w+2\mu)6\mu n^2\frac{\left(\alpha p^{\frac{1}{2^{d'}}c_2}\right)^{c_2 s}}{1-\left(\alpha p^{\frac{1}{2^{d'}}c_2}\right)} + ((w+2\mu)6\mu n^2)^2\left(\alpha p^{\frac{1}{2^{d'+1}}c_1}\right)^{\frac{1}{2d}n^{\frac{1}{d}}}$$

□

Now, combined with Claim 3.15, with a probability greater than:

$$1 - \Theta\left(((w + 2\mu)\mu n^2)^3\right)(\alpha p^{c_7})^{\gamma' n^{\frac{1}{d}}}$$

No error cluster has a size greater than $\gamma n^{\frac{1}{d}}$, so the decoder successfully restores the logical state; in other words, no logical error occurs. □

Theorem 3.87. *There exists a constant $p_0 \in (0, 1)$ such that for any local stochastic noise channel $\mathcal{E}$ with rate $p < p_0$, and a quantum circuit C with depth μ and width w, there exists a toric encoded circuit $\tilde{C}$, with depth μn and width $wn + 2\mu n$ that simulates C with probability:*

References

[Neu56] J. von Neumann. "Probabilistic Logics and the Synthesis of Reliable Organisms From Unreliable Components". In: *Automata Studies*. Ed. by C. E. Shannon and J. McCarthy. Princeton: Princeton University Press, 1956, pp. 43–98. ISBN: 9781400882618. DOI: doi:10.1515/9781400882618-003. URL: https://doi.org/10.1515/9781400882618-003.

[BS88] Piotr Berman and J'anos Simon. "Investigations of fault-tolerant networks of computers". In: *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*. STOC '88. Chicago, Illinois, USA: Association for Computing Machinery, 1988, pp. 66–77. ISBN: 0897912640. DOI: 10.1145/62212.62219. URL: https://doi.org/10.1145/62212.62219.

[AB99] Dorit Aharonov and Michael Ben-Or. *Fault-Tolerant Quantum Computation With Constant Error Rate*. 1999. arXiv: quant-ph/9906129 [quant-ph].

[ZLC00] Xinlan Zhou, Debbie W. Leung, and Isaac L. Chuang. "Methodology for quantum logic gate construction". In: *Physical Review A* 62.5 (Oct. 2000). ISSN: 1094-1622. DOI: 10.1103/physreva.62.052316. URL: http://dx.doi.org/10.1103/PhysRevA.62.052316.

[Den+02] Eric Dennis et al. "Topological quantum memory". In: *Journal of Mathematical Physics* 43.9 (Sept. 2002), pp. 4452–4505. DOI: 10.1063/1.1499754. URL: https://doi.org/10.1063%2F1.1499754.

[Ali+08] R. Alicki et al. *On thermal stability of topological qubit in Kitaev's 4D model*. 2008. arXiv: 0811.0033 [quant-ph]. URL: https://arxiv.org/abs/0811.0033.

[Got14] Daniel Gottesman. *Fault-Tolerant Quantum Computation with Constant Overhead*. 2014. arXiv: 1310.2984 [quant-ph].

[Bom15] Héctor Bombín. "Gauge color codes: optimal transversal gates and gauge fixing in topological stabilizer codes". In: *New Journal of Physics* 17.8 (Aug. 2015), p. 083002. DOI: 10.1088/1367-2630/17/8/083002. URL: https://doi.org/10.1088/1367-2630/17/8/083002.

[FGL18] Omar Fawzi, Antoine Grospellier, and Anthony Leverrier. "Constant Overhead Quantum Fault-Tolerance with Quantum Expander Codes". In: *2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, Oct. 2018, pp. 743–754. DOI: 10.1109/focs.2018.00076. URL: http://dx.doi.org/10.1109/FOCS.2018.00076.

[Gro19] Antoine Grospellier. "Constant time decoding of quantum expander codes and application to fault-tolerant quantum computation". Theses. Sorbonne Université, Nov. 2019. URL: https://theses.hal.science/tel-03364419.

[KP19] Aleksander Kubica and John Preskill. "Cellular-Automaton Decoders with Provable Thresholds for Topological Codes". In: *Physical Review Letters* 123.2 (July 2019). ISSN: 1079-7114. DOI: 10.1103/physrevlett.123.020501. URL: http://dx.doi.org/10.1103/PhysRevLett.123.020501.

[Gac21] Peter Gacs. *A new version of Toom's proof*. 2021. arXiv: 2105.05968 [cs.FL]. URL: https://arxiv.org/abs/2105.05968.

[BB24] Nikolas P. Breuckmann and Simon Burton. "Fold-Transversal Clifford Gates for Quantum Codes". In: *Quantum* 8 (2024), p. 1372. ISSN: 2521-327X. DOI: 10.22331/q-2024-06-13-1372. URL: http://dx.doi.org/10.22331/q-2024-06-13-1372.

[GV24] Alexandre Guernut and Christophe Vuillot. *Fault-Tolerant Constant-Depth Clifford Gates on Toric Codes*. 2024. arXiv: 2411.18287 [quant-ph]. URL: https://arxiv.org/abs/2411.18287.

[NP25] Quynh T. Nguyen and Christopher A. Pattison. *Quantum fault tolerance with constant-space and logarithmic-time overheads*. 2025. arXiv: 2411.03632 [quant-ph]. URL: https://arxiv.org/abs/2411.03632.

[BDL26] Shankar Balasubramanian, Margarita Davydova, and Ethan Lake. "A local automaton for the 2D toric code". In: *Quantum* 10 (2026), p. 2125. ISSN: 2521-327X. DOI: 10.22331/q-2026-06-02-2125. URL: http://dx.doi.org/10.22331/q-2026-06-02-2125.

[DST26] Gesa Dünnweber, Georgios Styliaris, and Rahul Trivedi. *Quantum Memory and Autonomous Computation in Two Dimensions*. 2026. arXiv: 2601.20818 [quant-ph]. URL: https://arxiv.org/abs/2601.20818.

[SST26] Jan M. Swart, Réka Szabó, and Cristina Toninelli. *Peierls bounds from Toom contours*. 2026. arXiv: 2202.10999 [math.PR]. URL: https://arxiv.org/abs/2202.10999.